📂 CATEGORY 5: Networking — Complete Deep Dive

Linux & Bash Mastery — DevOps/SRE/Platform Engineer Interview Guide

---

Table of Contents

- 5.1 ping, traceroute, dig, nslookup — Basic Connectivity
- 5.2 netstat / ss — Socket States, Listening Ports
- 5.3 curl & wget — HTTP from the Command Line
- 5.4 tcpdump — Packet Capture
- 5.5 iptables / nftables — Firewall Rules, Chains, NAT
- 5.6 Network Interfaces — ip addr, ip route, ip link
- 5.7 DNS Resolution Chain — /etc/resolv.conf, /etc/hosts, nsswitch.conf
- 5.8 ssh Internals — Key Exchange, Tunnels, ~/.ssh/config, ProxyJump
- 5.9 TCP States — SYN_SENT, TIME_WAIT, CLOSE_WAIT
- 5.10 Network Namespaces — Container Isolation
- 5.11 eBPF Basics — Modern Linux Networking Observability
- Master Gotcha List
- Top Interview Questions

5.1 `ping`, `traceroute`, `dig`, `nslookup` — Basic Connectivity

🔷 What they are in simple terms

These are your **first-response tools** when something is broken on the network. Before reaching for anything complex, every SRE runs these four in sequence to narrow down where a problem lives.

🔷 `ping` — Is the host reachable?

How it works internally

```
Your machine                    Target host
     |                                |
     |── ICMP Echo Request ──────────►|
     |                                |  (kernel responds automatically)
     |◄─ ICMP Echo Reply ─────────────|
     |                                |
  RTT measured
  (Round Trip Time)
```

`ping` sends **ICMP Echo Request** packets. The target's kernel (not any application) responds with **ICMP Echo Reply**. This means ping works even when all application services are down — it tests Layer 3 (IP) reachability only.

```
# Basic ping
ping google.com
# PING google.com (142.250.80.46): 56 data bytes
# 64 bytes from 142.250.80.46: icmp_seq=0 ttl=116 time=12.4 ms

# -c: count (stop after N packets)
ping -c 4 google.com

# -i: interval between packets (default 1s)
ping -i 0.2 google.com    # Fast ping (200ms intervals)

# -s: packet size (default 56 bytes data = 64 bytes with ICMP header)
ping -s 1472 google.com   # Test MTU (1472 + 28 = 1500 byte MTU)

# -W: timeout per packet (seconds)
ping -c 1 -W 2 192.168.1.1  # Fail fast — 2 second timeout

# -t: TTL (Time To Live)
ping -t 5 google.com      # Only packets that reach ≤5 hops

# Flood ping (requires root) — stress test
sudo ping -f -c 10000 target   # Sends as fast as possible

# ── Reading ping output ──────────────────────────────────────────────
# 64 bytes from 1.1.1.1: icmp_seq=1 ttl=56 time=8.23 ms
#                                   ─────    ──────   ────────────
#                                   TTL      hops     round-trip time
# ttl=56 means started at 64, crossed 8 hops  (64-56=8)
# ttl=116 means started at 128, crossed 12 hops (128-116=12)

# Summary line:
# 4 packets transmitted, 4 received, 0% packet loss, time 3004ms
# rtt min/avg/max/mdev = 8.1/9.2/11.4/1.3 ms
#                                          ── mdev = jitter

# Production use: check if host is alive in a script
if ping -c 1 -W 2 "$HOST" &>/dev/null; then
    echo "$HOST is reachable"
else
    echo "$HOST is DOWN" >&2
fi
```

🔷 `traceroute` — Where does the path break?

How it works internally

```
TTL Trick:
┌──────────────────────────────────────────────────────────────┐
│ Probe 1: TTL=1 → Router1 decrements to 0 → ICMP              │
```

```
│          Time Exceeded → Router1 reveals its IP          │
│                                                          │
│ Probe 2: TTL=2 → Router1 forwards → Router2 decrements│
│          to 0 → ICMP Time Exceeded → Router2 revealed   │
│                                                          │
│ Probe N: TTL=N → packet reaches destination →            │
│          ICMP Port Unreachable → done                    │
└──────────────────────────────────────────────────────────┘
```

```
# Basic traceroute
traceroute google.com
# traceroute to google.com (142.250.80.46), 30 hops max
#  1  192.168.1.1   0.8 ms   0.9 ms   0.7 ms   ← your gateway
#  2  10.0.0.1      5.2 ms   5.1 ms   5.3 ms   ← ISP router
#  3  * * *                                    ← ICMP blocked!
#  4  142.250.80.46 12.4 ms  12.1 ms  12.6 ms  ← destination

# -n: don't resolve hostnames (much faster)
traceroute -n google.com

# -T: use TCP SYN instead of UDP (bypasses many firewalls)
sudo traceroute -T -p 443 google.com

# -I: use ICMP Echo (like ping, not UDP)
sudo traceroute -I google.com

# -m: max hops (default 30)
traceroute -m 15 google.com

# mtr — real-time traceroute (best of ping + traceroute combined)
mtr google.com                      # Interactive live view
mtr --report -c 100 google.com      # 100 samples, then report

# mtr output:
# Host              Loss%  Snt  Last  Avg  Best  Wrst  StDev
# 1. 192.168.1.1    0.0%   100  0.8   0.9  0.7   2.1   0.1
# 2. 10.0.0.1       0.0%   100  5.1   5.2  4.9   7.3   0.3
# 3. ???           100.0%  100  0.0   0.0  0.0   0.0   0.0  ← ICMP blocked
# 4. 142.250.80.46  0.0%   100  12.1  12.3 11.8  14.2  0.4

# Reading traceroute:
# * * *     = router dropped ICMP (not necessarily broken!)
# High RTT at hop N, low at N+1 = ICMP deprioritized at N (normal)
# High RTT from hop N ONWARDS   = real bottleneck IS at hop N
```

🔷 `dig` — DNS lookup tool (THE one to use)

How DNS resolution works

```
Application asks: "What is the IP of api.example.com?"
         │
         ▼
┌─────────────────┐
│  /etc/hosts     │  ← checked first (configurable via nsswitch.conf)
└─────────────────┘
         │ Not found
         ▼
┌──────────────────┐
│  /etc/resolv.conf  │  ← gets the DNS server IP (e.g., 8.8.8.8)
│  nameserver        │
└──────────────────┘
         │
         ▼
┌──────────────────────────────────────────────────┐
│  Recursive DNS Resolver (your configured nameserver) │
│  (e.g., 8.8.8.8 or your VPC DNS at 169.254.169.253) │
└──────────────────────────────────────────────────┘
         │ Queries upstream if not cached
         ▼
    Root Nameservers (.com? → go to .com TLD servers)
         │
         ▼
    TLD Nameservers (.com → go to example.com nameservers)
         │
         ▼
    Authoritative Nameserver (example.com → 93.184.216.34)
```

```
# Basic lookup (A record — IPv4)
dig google.com
# ;; ANSWER SECTION:
# google.com.    299    IN    A    142.250.80.46
#                ───         —    ──────────────
#                TTL         type  IP address

# Specific record types
dig google.com A       # IPv4 address
dig google.com AAAA    # IPv6 address
dig google.com MX      # Mail servers
dig google.com NS      # Nameservers
dig google.com TXT     # Text records (SPF, DKIM, verification)
dig google.com CNAME   # Canonical name alias
dig google.com SOA     # Start of Authority (zone info)

# +short: minimal output — perfect for scripts
dig +short google.com           # 142.250.80.46
dig +short google.com MX        # 10 smtp.google.com.
dig +short google.com NS        # ns1.google.com. ns2.google.com.

# Query specific DNS server (bypass /etc/resolv.conf)
dig @8.8.8.8 google.com         # Query Google's DNS
```

```
dig @1.1.1.1 google.com         # Query Cloudflare's DNS
dig @192.168.1.1 internal.corp  # Query internal DNS server

# Reverse DNS lookup (PTR record)
dig -x 8.8.8.8                  # What hostname is 8.8.8.8?
dig -x 8.8.8.8 +short           # dns.google.

# Trace full resolution path
dig +trace google.com            # Shows every step from root → answer

# Check DNS propagation (compare authoritative vs cached)
dig +norecurse @8.8.8.8 google.com   # Ask without recursion

# DNSSEC validation
dig +dnssec google.com

# Production patterns:
# Check if DNS record exists and matches expected value
ACTUAL=$(dig +short api.myapp.com)
EXPECTED="10.0.0.50"
[[ "$ACTUAL" == "$EXPECTED" ]] || echo "DNS mismatch! Got: $ACTUAL"

# Wait for DNS propagation after a change
until [[ "$(dig +short @8.8.8.8 "$DOMAIN")" == "$NEW_IP" ]]; do
    echo "Waiting for DNS propagation..."
    sleep 30
done
echo "DNS propagated!"
```

🔷 `nslookup` — Simpler DNS tool (legacy but common in interviews)

```
# Interactive mode
nslookup
> google.com
> set type=MX
> google.com
> exit

# Non-interactive
nslookup google.com              # Default lookup
nslookup google.com 8.8.8.8      # Using specific server
nslookup -type=MX google.com     # Specific record type
nslookup -type=PTR 8.8.8.8       # Reverse lookup

# ⚠ dig vs nslookup:
# dig is preferred — more detailed, scriptable, standards-compliant
# nslookup is simpler but output format can vary between implementations
# In interviews: say you prefer dig but know nslookup for quick checks
```

🔷 Short crisp interview answer

> "My connectivity debugging order is: `ping` to check Layer 3 reachability via ICMP, `traceroute -n` (with -n to skip DNS) to find where packets stop, and `dig` for DNS issues. `ping` failure doesn't always mean the host is down — ICMP may be blocked. `traceroute` works by incrementing TTL from 1 upward and collecting ICMP Time Exceeded responses from each hop. For DNS, `dig @8.8.8.8 domain.com +short` lets me query a specific server to isolate whether the issue is my resolver or the authoritative nameserver."

⚠ Tricky gotchas

```
# GOTCHA 1: ping failure ≠ host down
# Many cloud services (AWS, Azure) block ICMP by default
# A host can be serving HTTP fine but not respond to ping
# Always verify with: nc -zv host 443  or  curl -I https://host

# GOTCHA 2: traceroute * * * doesn't mean broken
# Many routers silently drop ICMP TTL-exceeded messages
# The path continues — subsequent hops may respond fine

# GOTCHA 3: traceroute high latency at one hop
# If hop N shows 200ms but hop N+1 shows 12ms:
# → Router at N is just deprioritizing ICMP responses
# → Actual traffic flows through N at normal speed

# GOTCHA 4: dig TTL meaning
dig google.com
# TTL=299 means: this answer is cached, expires in 299 seconds
# After TTL expires, resolver re-queries the authoritative server

# GOTCHA 5: dig response codes
# NOERROR  + ANSWER section    = record found
# NOERROR  + empty ANSWER      = record type doesn't exist
# NXDOMAIN                     = domain definitely doesn't exist
# SERVFAIL                     = DNS server couldn't answer
(config/network issue)
```

5.2 `netstat` / `ss` — Socket States, Listening Ports

🔷 What they are

`netstat` and `ss` show you **all network connections and listening ports** on a system. `ss` is the modern replacement — it reads directly from the kernel's socket tables via netlink, making it faster and more detailed than `netstat` which reads from `/proc`.

```
┌──────────────────────────────────────────────────────────────┐
│                    Kernel Socket Table                       │
│  Proto  LocalAddr      RemoteAddr      State    PID/Prog     │
│  tcp    0.0.0.0:80     0.0.0.0:*       LISTEN   nginx        │
│  tcp    10.0.0.1:80    1.2.3.4:54321   ESTABLISHED nginx     │
│  tcp    10.0.0.1:443   5.6.7.8:61234   ESTABLISHED nginx     │
└──────────────────────────────────────────────────────────────┘

         ▲
         │  ss reads directly from kernel (netlink socket)
         │  netstat reads from /proc/net/tcp (slower)
```

🔷 `ss` — The modern tool (prefer this)

```
# The flag alphabet — memorize this combination: ss -tulnp
# -t  TCP sockets
# -u  UDP sockets
# -l  Listening sockets only
# -n  Numeric (don't resolve names) — faster
# -p  Show process using the socket

# Most used: show all listening TCP/UDP ports with process
ss -tulnp

# Output:
# Netid  State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
Process
# tcp    LISTEN  0       128     0.0.0.0:22          0.0.0.0:*
users:(("sshd",pid=1234,fd=3))
# tcp    LISTEN  0       511     0.0.0.0:80          0.0.0.0:*
users:(("nginx",pid=5678,fd=6))
# tcp    LISTEN  0       128     127.0.0.1:5432      0.0.0.0:*
users:(("postgres",pid=910,fd=5))

# All established connections
ss -tn state established

# Show connections to specific port
ss -tn dst :443

# Show connections FROM specific IP
ss -tn src 10.0.0.1

# Filter by state
ss -t state time-wait        # All TIME_WAIT connections
ss -t state close-wait       # All CLOSE_WAIT connections

# Count connections per state
ss -t | awk '{print $1}' | sort | uniq -c | sort -rn
```

```
# Show socket memory usage
ss -m

# Extended info (timers, options)
ss -tne

# Watch connections in real-time
watch -n 1 'ss -tn | awk "{print \$1}" | sort | uniq -c'

# Find what's using port 8080
ss -tlnp | grep :8080

# Full connection detail — source IP, dest IP, port, state, process
ss -tnp
# ESTAB  0  0  10.0.0.1:80  192.168.1.5:54321  users:(("nginx",pid=123))
```

🔷 `netstat` — Legacy but still asked about

```
# netstat flags mirror ss flags
netstat -tulnp    # Same output as ss -tulnp

# All connections
netstat -an

# Routing table
netstat -rn       # Same as: ip route show

# Interface statistics
netstat -i

# Network protocol statistics
netstat -s        # Detailed: TCP retransmits, errors, etc.

# ⚠ netstat is deprecated on modern Linux
# Install if missing: sudo apt install net-tools
# Prefer ss — it's faster and part of iproute2
```

🔷 Reading the output — socket states

```
LISTEN       → Server waiting for connections (called listen())
ESTABLISHED  → Active connection, data flowing
TIME_WAIT    → Connection closed, waiting for delayed packets (2×MSL)
CLOSE_WAIT   → Remote side closed, local app hasn't closed yet
SYN_SENT     → Client sent SYN, waiting for SYN-ACK
SYN_RECV     → Server received SYN, sent SYN-ACK, waiting for ACK
FIN_WAIT_1   → Local side initiated close, sent FIN
FIN_WAIT_2   → Local FIN acknowledged, waiting for remote FIN
```

```
CLOSING      → Both sides sent FIN simultaneously
LAST_ACK     → Remote closed, local closed, waiting for final ACK
```

🔷 Production use cases

```
# Incident: "What is listening on port 443?"
ss -tlnp | grep :443

# Incident: "How many connections does nginx have right now?"
ss -tn state established | grep :80 | wc -l

# Incident: "Are we being SYN flooded?"
ss -tn state syn-recv | wc -l    # Large number = possible SYN flood

# Incident: "Why is the app slow?" — check socket backlogs
ss -tlnp
# Recv-Q non-zero on LISTEN socket = app not accepting connections fast
enough!
# Recv-Q non-zero on ESTABLISHED   = data received but app not reading it

# Incident: "Too many TIME_WAIT connections"
ss -t state time-wait | wc -l
# High TIME_WAIT is normal under load (2 min expiry per connection)
# Fix: enable tcp_tw_reuse, or connection pooling in the application

# Monitoring: connections per remote IP
ss -tn state established | awk '{print $5}' | cut -d: -f1 | sort | uniq -c
| sort -rn | head

# Verify service is actually listening (not just port-forwarded)
ss -tlnp | grep -E ':80|:443'
```

🔷 Short crisp interview answer

> "`ss -tulnp` is my first command when debugging connectivity — it shows all listening TCP/UDP ports with the process name and PID. `ss` reads directly from the kernel's netlink socket so it's faster than `netstat` which reads from `/proc`. I check the Recv-Q column on LISTEN sockets — if it's non-zero, the application can't accept connections fast enough. For established connections I look for unusual state counts: excessive TIME_WAIT is normal under load, but many CLOSE_WAIT means the application isn't closing connections properly — which is always an app bug."

⚠ Tricky gotchas

```
# GOTCHA 1: Recv-Q meaning differs by socket state
# On LISTEN socket:    Recv-Q = connections waiting to be accept()ed
```

```
(backlog)
# On ESTABLISHED:      Recv-Q = bytes received but not yet read by
application

# GOTCHA 2: 0.0.0.0:80 vs 127.0.0.1:80
# 0.0.0.0:80    = listening on ALL interfaces (accessible from network)
# 127.0.0.1:80  = listening on loopback ONLY (local machine only)
# :::80         = listening on all IPv6 interfaces (usually includes IPv4
too)

# GOTCHA 3: ss -p requires root for other processes' socket info
ss -tlnp         # Shows your own processes only
sudo ss -tlnp    # Shows ALL processes — required for full visibility

# GOTCHA 4: Port "listening" doesn't mean service is healthy
# A process can listen on a port but be deadlocked or in a broken state
```

5.3 `curl` & `wget` — HTTP from the Command Line

🔷 What they are

`curl` (**C**lient **URL**) and `wget` are command-line HTTP clients. `curl` is the DevOps workhorse — it supports dozens of protocols and gives you precise control over every aspect of an HTTP request. `wget` is simpler — best for downloading files recursively.

🔷 `curl` — The complete toolkit

```
# Basic GET request
curl https://api.example.com/users

# The four most important flags for scripts:
# -s: silent (no progress bar)
# -S: show errors even with -s
# -f: fail on HTTP errors (exit code 22 on 4xx/5xx) ← critical for
scripts!
# -L: follow redirects
curl -sSfL https://api.example.com/health

# ── Response inspection ──────────────────────────────────────────

# Show response headers only (HEAD request)
curl -I https://api.example.com

# Show request AND response headers
curl -v https://api.example.com          # Verbose — shows everything

# Even more detail (SSL handshake, TLS version, etc.)
curl -vvv https://api.example.com
```

```
# Show only HTTP status code
curl -o /dev/null -s -w "%{http_code}" https://api.example.com

# Show timing breakdown — critical for performance debugging
curl -o /dev/null -s -w "
  DNS lookup:     %{time_namelookup}s
  TCP connect:    %{time_connect}s
  SSL handshake:  %{time_appconnect}s
  TTFB:           %{time_starttransfer}s
  Total:          %{time_total}s
  HTTP status:    %{http_code}
" https://api.example.com

# ── HTTP methods ──────────────────────────────────────────────

# POST with JSON body
curl -X POST https://api.example.com/users \
     -H "Content-Type: application/json" \
     -d '{"name": "Alice", "email": "alice@example.com"}'

# POST with form data
curl -X POST https://api.example.com/login \
     -d "username=alice&password=secret"

# POST from file
curl -X POST https://api.example.com/upload \
     -H "Content-Type: application/json" \
     -d @payload.json

# PUT
curl -X PUT https://api.example.com/users/123 \
     -H "Content-Type: application/json" \
     -d '{"name": "Alice Updated"}'

# DELETE
curl -X DELETE https://api.example.com/users/123

# PATCH
curl -X PATCH https://api.example.com/users/123 \
     -H "Content-Type: application/json" \
     -d '{"email": "newemail@example.com"}'

# ── Authentication ────────────────────────────────────────────

# Basic auth
curl -u username:password https://api.example.com

# Bearer token
curl -H "Authorization: Bearer eyJhbGciOiJSUzI1NiJ9..."
https://api.example.com

# API key in header
curl -H "X-API-Key: abc123def456" https://api.example.com
```

```
# API key in query string
curl "https://api.example.com/data?api_key=abc123"

# Client certificate (mTLS)
curl --cert client.crt --key client.key https://api.example.com

# ── Headers ──────────────────────────────────────────────────────

# Custom headers
curl -H "Accept: application/json" \
     -H "X-Request-ID: $(uuidgen)" \
     -H "User-Agent: MyApp/1.0" \
     https://api.example.com

# ── SSL/TLS ──────────────────────────────────────────────────────

# Skip certificate verification (NEVER in production!)
curl -k https://self-signed.example.com

# Use custom CA certificate
curl --cacert /path/to/ca.pem https://internal.example.com

# Check SSL certificate info
curl -vI https://api.example.com 2>&1 | grep -A5 "SSL"

# ── Output control ───────────────────────────────────────────────

# Save to file
curl -o output.json https://api.example.com/data
curl -O https://example.com/file.tar.gz    # Use remote filename

# Resume interrupted download
curl -C - -O https://example.com/bigfile.tar.gz

# ── Timeout control ──────────────────────────────────────────────
curl --connect-timeout 5 \    # Max time to establish connection
     --max-time 30 \          # Max TOTAL time including transfer
     https://api.example.com

# ── Production patterns ──────────────────────────────────────────

# Health check in a script
health_check() {
    local url=$1
    local status
    status=$(curl -o /dev/null -s -w "%{http_code}" --max-time 5 "$url")
    if [[ "$status" == "200" ]]; then
        return 0
    else
        echo "Health check failed: HTTP $status" >&2
        return 1
    fi
}
```

```
# Wait for service to become ready (deployment scripts)
until curl -sf --max-time 2 http://localhost:8080/health; do
    echo "Waiting for service..."
    sleep 2
done

# Download with retry
curl --retry 3 --retry-delay 2 --retry-connrefused \
     -sSfL https://releases.example.com/app-v1.0.tar.gz \
     -o app.tar.gz

# POST JSON and parse response with jq
RESPONSE=$(curl -sSf \
    -X POST \
    -H "Content-Type: application/json" \
    -d '{"action": "deploy", "env": "production"}' \
    https://api.example.com/deploy)
JOB_ID=$(echo "$RESPONSE" | jq -r '.job_id')
echo "Deploy job: $JOB_ID"

# Rate-limited API calls
for id in "${IDS[@]}"; do
    curl -sS "https://api.example.com/items/$id" >> results.json
    sleep 0.1   # 10 req/sec rate limit
done
```

🔷 `wget` — Best for recursive downloads

```
# Basic download
wget https://example.com/file.tar.gz

# Download quietly (no output)
wget -q https://example.com/file.tar.gz

# Recursive website download
wget -r -l 2 https://example.com   # Recurse 2 levels deep

# Mirror a website
wget --mirror --convert-links --page-requisites https://example.com

# Download to specific file
wget -O output.tar.gz https://example.com/file.tar.gz

# Resume download
wget -c https://example.com/bigfile.tar.gz

# With authentication
wget --user=alice --password=secret https://secure.example.com/file
```

```
# Run in background
wget -b https://example.com/bigfile.tar.gz

# wget vs curl — when to use which:
# wget: recursive downloads, mirroring websites, simple file download
# curl: API calls, custom headers, auth, JSON, precise HTTP control
```

🔷 Short crisp interview answer

> "`curl -sSfL` is my go-to for API calls in scripts: `-s` suppresses progress, `-S` still shows errors, `-f` returns a non-zero exit code on HTTP 4xx/5xx, `-L` follows redirects. For debugging I use `-v` to see full headers and the TLS handshake, or `-w` with format strings to measure timing breakdown — DNS lookup, TCP connect, SSL handshake, and time-to-first-byte separately. In health check scripts I capture the HTTP status code with `-o /dev/null -w '%{http_code}'` and compare it to 200."

⚠️ Tricky gotchas

```
# GOTCHA 1: curl exits 0 even on 404/500 without -f!
curl https://api.example.com/notfound
echo $?    # 0 — even if HTTP 404!
curl -f https://api.example.com/notfound
echo $?    # 22 — now correctly fails and can be caught by set -e

# GOTCHA 2: -d sends Content-Type: application/x-www-form-urlencoded
# For JSON you MUST set the Content-Type header explicitly
curl -d '{"key":"val"}' https://api          # ❌ Wrong content type sent
curl -H "Content-Type: application/json" \
     -d '{"key":"val"}' https://api          # ✅ Correct

# GOTCHA 3: curl -k disables ALL certificate verification
# Never use -k in production — use --cacert for custom CAs instead

# GOTCHA 4: --max-time vs --connect-timeout
--connect-timeout 5   # Max time to ESTABLISH the connection
--max-time 30         # Max TOTAL time including data transfer
# In scripts: set BOTH to prevent indefinite hanging

# GOTCHA 5: Large response bodies go to terminal
curl https://api.example.com/huge-dataset  # Prints entire GB to terminal!
curl -o /dev/null https://...              # Discard body
curl -o output.json https://...           # Save to file
```

5.4 `tcpdump` — Packet Capture 🟡

🔷 What it is in simple terms

`tcpdump` captures **raw network packets** at the kernel level and displays or saves them. It's a network microscope — when you need to see exactly what's going over the wire, byte by byte.

🔷 How it works internally

```
Network Interface (eth0)
        │
        │ Packets flow through
        ▼
┌──────────────────┐
│  Kernel BPF      │ ←── Berkeley Packet Filter
│  (in-kernel      │     compiled filter runs here
│   packet filter) │     at interrupt time — very fast!
└──────────────────┘
        │  Matching packets only
        ▼
┌──────────────────┐
│  tcpdump userspace│
│  (libpcap)       │
└──────────────────┘
        │
        │
    ┌───┴──────────────┐
    │                  │
Display to terminal  Save to .pcap file
(human readable)     (Wireshark can read)
```

🔷 Core usage

```
# Basic — capture all packets on default interface
sudo tcpdump

# -i: interface
sudo tcpdump -i eth0          # Specific interface
sudo tcpdump -i any           # ALL interfaces

# -n: don't resolve hostnames (faster, clearer IPs)
# -nn: don't resolve hostnames OR port names
sudo tcpdump -nn -i eth0

# -v, -vv, -vvv: verbosity levels
sudo tcpdump -vnn -i eth0

# -c: capture N packets then stop
sudo tcpdump -c 100 -i eth0

# -w: write to file (for Wireshark or offline analysis)
sudo tcpdump -w /tmp/capture.pcap -i eth0
```

```
# -r: read from saved file
sudo tcpdump -r /tmp/capture.pcap

# -s: snap length (bytes per packet to capture)
sudo tcpdump -s 0             # Full packet (0 = unlimited, up to 65535)
sudo tcpdump -s 96            # First 96 bytes only (headers — saves
storage)

# Show packet contents in ASCII
sudo tcpdump -A -i eth0

# Show in hex AND ASCII
sudo tcpdump -XX -i eth0
```

🔷 BPF Filters — the real power

```
# ── By host ──────────────────────────────────────────────────────
sudo tcpdump host 192.168.1.1         # To OR from this host
sudo tcpdump src host 192.168.1.1     # FROM this host only
sudo tcpdump dst host 192.168.1.1     # TO this host only

# ── By port ──────────────────────────────────────────────────────
sudo tcpdump port 80                  # Port 80 traffic (either direction)
sudo tcpdump port 443                 # HTTPS traffic
sudo tcpdump dst port 80              # Traffic going TO port 80

# ── By protocol ──────────────────────────────────────────────────
sudo tcpdump tcp                      # TCP only
sudo tcpdump udp                      # UDP only
sudo tcpdump icmp                     # ICMP only (ping traffic)

# ── Combinations with and/or/not ─────────────────────────────────
sudo tcpdump -nn 'host 10.0.0.1 and port 443'
sudo tcpdump -nn 'port 80 or port 443'
sudo tcpdump -nn 'not port 22'            # Exclude SSH (reduces noise)
sudo tcpdump -nn 'host 10.0.0.1 and not icmp'

# ── Network/subnet ───────────────────────────────────────────────
sudo tcpdump net 10.0.0.0/24              # Entire subnet

# ── TCP flags ────────────────────────────────────────────────────
sudo tcpdump 'tcp[tcpflags] & tcp-syn != 0'    # SYN packets (new
connections)
sudo tcpdump 'tcp[tcpflags] & tcp-rst != 0'    # RST packets (connection
resets!)
sudo tcpdump 'tcp[tcpflags] == tcp-syn'         # ONLY SYN (not SYN-ACK)

# ── Packet size ──────────────────────────────────────────────────
sudo tcpdump 'len > 1000'                 # Large packets only
```

🔷 Real production examples

```
# Scenario 1: Debug HTTP requests to your app
sudo tcpdump -nn -A -i eth0 'port 80 and host 10.0.0.5'

# Scenario 2: See ALL DNS queries being made by the server
sudo tcpdump -nn -i any 'port 53'

# Scenario 3: Capture HTTPS handshake (see TLS Client Hello metadata)
sudo tcpdump -nn -i eth0 'port 443 and tcp[tcpflags] & tcp-syn != 0'

# Scenario 4: Find connection resets (RST = abrupt close)
sudo tcpdump -nn -i eth0 'tcp[tcpflags] & tcp-rst != 0'
# Many RSTs = firewall dropping packets, app crashing, or connection
timeout

# Scenario 5: Capture to file during incident, analyze later
sudo tcpdump -nn -i eth0 \
    -w /tmp/incident_$(date +%Y%m%d_%H%M%S).pcap \
    'not port 22' &    # Background, exclude SSH noise
TCPDUMP_PID=$!
# ...reproduce the issue...
sudo kill $TCPDUMP_PID     # Stop capture

# Read back selectively
sudo tcpdump -nn -r /tmp/incident_*.pcap 'port 443'

# Scenario 6: Find bandwidth hogs by host
sudo tcpdump -nn -i eth0 -l | awk '
/IP / {
    split($3, src, ".")
    ip = src[1]"."src[2]"."src[3]"."src[4]
    bytes[ip] += length($0)
}
END { for (h in bytes) print bytes[h], h }
' | sort -rn | head -10
```

🔷 Short crisp interview answer

> "`tcpdump` captures packets using BPF (Berkeley Packet Filter) — the filter runs in-kernel so only matching packets are copied to userspace, making it efficient even on high-traffic interfaces. I always use `-nn` to skip name resolution, `-w` to save to pcap for later Wireshark analysis, and BPF filters to reduce noise — especially `not port 22` to exclude SSH. For production incidents I capture to a file in the background while reproducing the issue, then analyze offline. Key filters: `port 443`, `host x.x.x.x`, `tcp[tcpflags] & tcp-rst != 0` for finding connection resets."

⚠ Tricky gotchas

```
# GOTCHA 1: tcpdump on HTTPS traffic shows encrypted data only
# You see: IPs, ports, packet sizes, TLS handshake metadata
# You CANNOT see: HTTP headers, URLs, request bodies (all encrypted)

# GOTCHA 2: Capturing on loopback requires explicit specification
sudo tcpdump -i lo    # Must specify lo for localhost traffic
# Traffic between local processes goes via lo, not eth0

# GOTCHA 3: Disk space — capture files grow fast!
sudo tcpdump -w capture.pcap   # Can fill disk on busy server!
# Add -C to rotate at file size:
sudo tcpdump -w capture.pcap -C 100   # New file every 100MB

# GOTCHA 4: Dropped packets under high load
# "X packets dropped by kernel" = capture buffer full
# Increase buffer: sudo tcpdump -B 65536 -i eth0 ...

# GOTCHA 5: tcpdump requires root or CAP_NET_RAW
tcpdump ...           # Permission denied
sudo tcpdump ...      # Works
```

5.5 `iptables` / `nftables` — Firewall Rules, Chains, NAT 🟡

🔷 What it is

`iptables` is the Linux firewall — it intercepts packets at kernel level and applies rules to accept, drop, reject, or modify them. `nftables` is its modern replacement (Linux 3.13+), but `iptables` is still dominant in production and heavily tested in interviews.

🔷 How it works internally — Netfilter hooks

```
Packet arrives on eth0
        │
        ▼
┌──────────────────────────────────────────────────────────┐
│                      NETFILTER HOOKS                     │
│                                                          │
│  ┌──────────────┐    ┌──────────┐    ┌──────────────────┐│
│  │  PREROUTING  │    │  INPUT   │    │     FORWARD      │
│  │(nat, mangle) │    │(filter)  │    │    (filter)      │
│  └──────────────┘    └──────────┘    └──────────────────┘
│         │                │                    │          │
│         ▼                │                    │          │
│   Routing Decision       │                    │          │
│  ┌──────────────────┐    │                    │          │
│  │ For this machine?│────┘                    │          │
│  │ For other host?  │──────────────────────────          │
```

```
│   └──────────────────┘                                          │
│          │                                                       │
│   ┌──────▼──────────┐     ┌──────────────────┐                  │
│   │    OUTPUT       │     │    POSTROUTING   │                  │
│   │   (filter)      │     │  (nat, mangle)   │                  │
│   └─────────────────┘     └──────────────────┘                  │
└──────────────────────────────────────────────────────────────────┘

Tables:
- filter  : packet filtering (ACCEPT/DROP/REJECT) — default table
- nat     : address translation (DNAT/SNAT/MASQUERADE)
- mangle  : packet modification (TTL, TOS, mark)
- raw     : connection tracking bypass
```

🔷 Core `iptables` usage

```
# ── Viewing rules ──────────────────────────────────────────────

# List all rules with line numbers and packet/byte counts
sudo iptables -L -n -v --line-numbers

# List specific chain
sudo iptables -L INPUT -n -v

# List NAT rules
sudo iptables -t nat -L -n -v

# ── Basic rules ────────────────────────────────────────────────

# Allow incoming SSH
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow incoming HTTP/HTTPS
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow established/related connections (CRITICAL — always add this
first!)
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow loopback interface
sudo iptables -A INPUT -i lo -j ACCEPT

# Set default deny (drop everything not explicitly allowed)
sudo iptables -P INPUT DROP

# Allow specific source IP only
sudo iptables -A INPUT -s 10.0.0.0/8 -j ACCEPT

# Block specific IP (DDoS mitigation)
```

```
sudo iptables -I INPUT 1 -s 1.2.3.4 -j DROP   # -I inserts at position 1
(first rule)

# DROP vs REJECT
sudo iptables -A INPUT -p tcp --dport 23 -j DROP    # Silently drop
(attacker doesn't know)
sudo iptables -A INPUT -p tcp --dport 23 -j REJECT  # Send RST/ICMP back
(polite refusal)

# ── Rule management ──────────────────────────────────────────

# Delete rule by line number
sudo iptables -D INPUT 3

# Delete rule by specification
sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT

# Insert rule at specific position
sudo iptables -I INPUT 1 -s 10.0.0.5 -j ACCEPT   # Becomes first rule

# Flush all rules in a chain
sudo iptables -F INPUT

# Flush ALL rules in ALL chains
sudo iptables -F

# Reset everything to default ACCEPT
sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT ACCEPT
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD ACCEPT

# ── NAT rules ────────────────────────────────────────────────

# MASQUERADE — SNAT for internet sharing
# All traffic from internal network appears as eth0's public IP
sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/8 -o eth0 -j MASQUERADE

# DNAT — forward incoming port 80 to internal server port 8080
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 \
    -j DNAT --to-destination 10.0.0.5:8080

# Port forwarding on same machine (8080 → 80)
sudo iptables -t nat -A OUTPUT -p tcp --dport 8080 \
    -j REDIRECT --to-ports 80

# ── Save and restore rules ───────────────────────────────────
sudo iptables-save > /etc/iptables/rules.v4         # Save to file
sudo iptables-restore < /etc/iptables/rules.v4      # Restore from file

# ── Rate limiting — SSH brute force protection ───────────────
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
    -m recent --set --name SSH
```

```
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
    -m recent --update --seconds 60 --hitcount 4 --name SSH -j DROP
# Allows 3 new SSH connections per minute per IP
```

🔷 `nftables` — the modern replacement

```
# nftables uses a single unified tool: nft
# Replaces iptables, ip6tables, arptables, ebtables in one framework

# List all rules
sudo nft list ruleset

# Basic firewall setup with nftables
sudo nft add table inet filter
sudo nft add chain inet filter input \
    '{ type filter hook input priority 0; policy drop; }'
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input iif lo accept
sudo nft add rule inet filter input tcp dport 22 accept
sudo nft add rule inet filter input tcp dport { 80, 443 } accept

# Check if nftables or iptables is active
sudo iptables -L    # If shows nft backend, system uses nftables under the
hood
```

🔷 Short crisp interview answer

> "iptables operates at the Netfilter kernel hooks. Packets traverse chains — PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING — and each rule either ACCEPTs, DROPs, or REJECTs the packet. Three tables handle different concerns: `filter` for accept/drop decisions, `nat` for address translation (DNAT for port forwarding, MASQUERADE for NAT/internet sharing), and `mangle` for packet modification. The critical rule everyone forgets is allowing ESTABLISHED,RELATED connections — without it, responses to your own outbound connections get dropped. `nftables` is the modern replacement with a single tool, cleaner syntax, and better performance."

⚠ Tricky gotchas

```
# GOTCHA 1: Rule ORDER matters — first match wins
sudo iptables -A INPUT -j DROP                          # Drop all
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT      # This NEVER runs!
# Fix: insert the ACCEPT rule BEFORE the DROP:
sudo iptables -I INPUT 1 -p tcp --dport 22 -j ACCEPT

# GOTCHA 2: Forgetting ESTABLISHED,RELATED rule
sudo iptables -P INPUT DROP
```

```
# Now you can't receive ANY responses to your own connections!
# Always add FIRST: iptables -A INPUT -m state --state ESTABLISHED,RELATED
-j ACCEPT

# GOTCHA 3: iptables rules are NOT persistent across reboots!
# Must save: sudo iptables-save > /etc/iptables/rules.v4
# And restore on boot via iptables-restore or ufw/firewalld

# GOTCHA 4: DROP vs REJECT behavior
# DROP: packet silently discarded — client hangs until timeout (default 2+
minutes)
# REJECT: sends RST (TCP) or ICMP Unreachable — client fails fast
# Use REJECT for legitimate services, DROP for known attackers

# GOTCHA 5: Flushing rules without resetting default policy
sudo iptables -F    # Flushes rules — but if policy was DROP, still blocks
everything!
sudo iptables -P INPUT ACCEPT   # Must also reset the policy
```

5.6 Network Interfaces — `ip addr`, `ip route`, `ip link` 🟡

🔷 The `ip` command — the modern tool

The `ip` command (from `iproute2`) replaces the deprecated `ifconfig`, `route`, and `arp` commands. Every modern Linux system uses it.

```
ip [object] [command]

Objects:
  addr    → IP addresses on interfaces
  link    → Network interface properties (up/down, MTU, MAC)
  route   → Routing table
  neigh   → ARP table (neighbor/MAC cache)
  rule    → Routing policy rules
  tunnel  → IP tunnels
  netns   → Network namespaces
```

🔷 `ip addr` — IP address management

```
# Show all interfaces and their IP addresses
ip addr show
ip addr          # Same (show is default)
ip a             # Short form

# Output explained:
# 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
```

```
#     link/loopback 00:00:00:00:00:00
#     inet 127.0.0.1/8 scope host lo
#
# 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
#     link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
#     inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
#     inet6 fe80::42:acff:fe11:2/64 scope link

# Show specific interface
ip addr show eth0
ip addr show dev eth0    # Same

# Add an IP address to an interface
sudo ip addr add 10.0.0.5/24 dev eth0

# Remove an IP address
sudo ip addr del 10.0.0.5/24 dev eth0

# Add secondary IP (multiple IPs on one interface)
sudo ip addr add 192.168.1.100/24 dev eth0 label eth0:0
```

🔷 `ip link` — Interface management

```
# Show all interfaces with MAC address, MTU, and state
ip link show
ip l             # Short form

# Bring interface up or down
sudo ip link set eth0 up
sudo ip link set eth0 down

# Change MTU (e.g., enable jumbo frames in datacenter)
sudo ip link set eth0 mtu 9000

# Change MAC address
sudo ip link set eth0 down
sudo ip link set eth0 address 02:00:00:00:00:01
sudo ip link set eth0 up

# Interface flags explained:
# UP          = interface is administratively enabled
# LOWER_UP    = physical link is detected (cable plugged in / link active)
# BROADCAST   = supports broadcast
# MULTICAST   = supports multicast
# PROMISC     = promiscuous mode (receives ALL packets, not just its own)

# Enable promiscuous mode (needed for tcpdump/packet capture on bridge)
sudo ip link set eth0 promisc on
```

🔷 `ip route` — Routing table

```
# Show routing table
ip route show
ip route
ip r             # Short form

# Output explained:
# default via 10.0.0.1 dev eth0 proto dhcp src 10.0.0.5 metric 100
# ───────     ────────     ────                ────────          ───
# catch-all   gateway      interface           source IP         priority
#
# 10.0.0.0/24 dev eth0 proto kernel scope link src 10.0.0.5
# ───────────      ────                           ────────
# local subnet     interface                      our IP
# (directly connected — no gateway needed)

# Add a static route via gateway
sudo ip route add 192.168.2.0/24 via 10.0.0.1

# Add a route via specific interface directly
sudo ip route add 192.168.2.0/24 dev eth1

# Add or change default gateway
sudo ip route add default via 10.0.0.1
sudo ip route replace default via 10.0.0.2    # Replace existing default

# Delete a route
sudo ip route del 192.168.2.0/24

# Test which route a packet would take — very useful for debugging!
ip route get 8.8.8.8
# 8.8.8.8 via 10.0.0.1 dev eth0 src 10.0.0.5 uid 1000
#         ────────────  ──────────             ─────────
#         gateway       interface              our source IP

# Multiple routing tables (policy routing)
ip rule show                    # Show routing policy rules
ip route show table all         # Show all routing tables
```

🔷 `ip neigh` — ARP table

```
# Show ARP table (who owns what IP on the local network)
ip neigh show
ip n

# Output explained:
# 10.0.0.1 dev eth0 lladdr 02:42:ac:11:00:01 REACHABLE
# ────────      ────        ─────────────────  ─────────
```

```
# IP           interface   MAC address         state

# ARP states:
# REACHABLE  = recently confirmed reachable
# STALE      = not confirmed recently, will probe before use
# DELAY      = waiting before sending probe
# PROBE      = actively probing
# FAILED     = unreachable
# PERMANENT  = static entry (never ages out)

# Flush ARP cache for an interface (if you suspect stale ARP)
sudo ip neigh flush dev eth0

# Add static ARP entry (permanent mapping)
sudo ip neigh add 10.0.0.100 lladdr 02:42:ac:11:00:05 dev eth0 nud
permanent
```

🔷 Bonding and VLANs

```
# ── VLAN (802.1Q tagging) ──────────────────────────────────────
# Create a VLAN subinterface (tags packets with VLAN ID 100)
sudo ip link add link eth0 name eth0.100 type vlan id 100
sudo ip link set eth0.100 up
sudo ip addr add 192.168.100.1/24 dev eth0.100

# ── Bonding (Link Aggregation / NIC teaming) ──────────────────
# Load bonding kernel module
sudo modprobe bonding

# Create bond interface
sudo ip link add bond0 type bond

# Set bonding mode (802.3ad = LACP — requires switch support)
sudo ip link set bond0 type bond mode 802.3ad

# Add physical interfaces as slaves to the bond
sudo ip link set eth0 master bond0
sudo ip link set eth1 master bond0

# Bring bond up and assign IP
sudo ip link set bond0 up
sudo ip addr add 10.0.0.5/24 dev bond0

# Check bond status
cat /proc/net/bonding/bond0

# Bond modes:
# mode 0: balance-rr     (round-robin)
# mode 1: active-backup  (failover — one active, one standby)
# mode 2: balance-xor    (XOR-based load balancing)
```

```
# mode 4: 802.3ad        (LACP — industry standard, needs switch config)
# mode 5: balance-tlb    (adaptive transmit load balancing)
# mode 6: balance-alb    (adaptive load balancing — best for most cases)
```

🔷 Short crisp interview answer

> "The `ip` command from iproute2 is the modern replacement for `ifconfig`, `route`, and `arp`. I use `ip addr show` for IP addresses, `ip link set up/down` for interface state, `ip route show` for the routing table, and critically `ip route get 8.8.8.8` to see exactly which route and source IP a packet would use — invaluable for debugging asymmetric routing. For the ARP table I use `ip neigh show` — stale ARP entries cause mysterious connectivity failures on the local network that look like routing problems. The `UP` flag means administratively enabled, but I need `LOWER_UP` to confirm the physical link is actually active."

⚠️ Tricky gotchas

```
# GOTCHA 1: UP flag vs physical link
ip link show eth0
# UP = interface is enabled in software
# LOWER_UP = physical layer is active (cable connected, link negotiated)
# Can have UP without LOWER_UP = enabled but no physical link!

# GOTCHA 2: ip commands are NOT persistent across reboots
# Use NetworkManager, /etc/network/interfaces, or netplan for persistence
ip addr add 10.0.0.5/24 dev eth0    # Lost after reboot!

# GOTCHA 3: ip route get shows the KERNEL decision, not just the table
# It accounts for policy routing rules, so it's the ground truth

# GOTCHA 4: Default route metric matters
# Lower metric = preferred route
# Two default routes? The lower metric one wins
ip route show | grep default    # Check which default route is active

# GOTCHA 5: ifconfig shows "incorrect" bytes after counter overflow
# ip -s link show eth0 uses 64-bit counters — use this instead
ip -s link show eth0    # Accurate statistics
```

5.7 DNS Resolution Chain — `/etc/resolv.conf`, `/etc/hosts`, `nsswitch.conf` 🟡

🔷 How name resolution works end-to-end

```
Application calls getaddrinfo("api.example.com")
                 |
```

```
                ▼
        ┌──────────────┐
        │ nsswitch.conf │    ← Defines the ORDER of lookup methods
        │ hosts: files  │        1. files  (/etc/hosts)
        │        dns    │        2. dns    (/etc/resolv.conf)
        └──────────────┘
                │
        ┌───────▼──────┐
        │  /etc/hosts   │    ← Check local file FIRST
        │  127.0.0.1 localhost
        │  10.0.0.5 db.internal
        └──────────────┘
                │ Not found
        ┌───────▼──────────────┐
        │  /etc/resolv.conf     │    ← Get DNS server address
        │  nameserver 8.8.8.8   │
        │  nameserver 8.8.4.4   │
        │  search corp.internal │
        │  options ndots:5      │
        └──────────────────────┘
                │
        DNS Query to 8.8.8.8:53
                │
                ▼
        Answer returned to application
```

🔷 `/etc/hosts` — Local override

```
cat /etc/hosts
# 127.0.0.1     localhost
# 127.0.1.1     myhostname
# ::1           localhost ip6-localhost ip6-loopback
# 10.0.0.5      db.internal db
# 10.0.0.6      api.internal api

# /etc/hosts OVERRIDES DNS by default (controlled by nsswitch.conf)

# Use cases in production:
# - Override DNS for testing (point prod domain to staging IP locally)
# - Speed up resolution for known hosts (skip DNS round-trip)
# - Block domains (point to 0.0.0.0 or 127.0.0.1)
# - Offline environments where DNS is unavailable

# In Docker containers:
# Docker injects the container's own hostname into /etc/hosts
automatically

# Quick host override for testing
echo "10.0.0.50  api.example.com" | sudo tee -a /etc/hosts
# Now api.example.com resolves to 10.0.0.50 on this machine only
```

🔷 `/etc/resolv.conf` — DNS client configuration

```
cat /etc/resolv.conf
# nameserver 8.8.8.8           ← Primary DNS server (queried first)
# nameserver 8.8.4.4           ← Secondary DNS (tried if primary fails)
# search corp.internal dev.internal  ← Search domains
# options ndots:5 timeout:2 attempts:3

# Directives explained:
# nameserver  : up to 3 allowed, tried in order on failure
# search      : domain suffixes appended to single-label queries
#               "dig database" → tries database.corp.internal first
# ndots       : if query has fewer dots than ndots, try search domains
first
#               ndots:5 means names with <5 dots try search domains first
# timeout     : seconds to wait for a response per server
# attempts    : how many times to retry each nameserver

# ⚠ On modern systems, resolv.conf is MANAGED automatically by:
# - systemd-resolved (Ubuntu 18.04+) — resolv.conf is a symlink!
# - NetworkManager — overwrites on network changes
# - cloud-init — sets on boot from cloud metadata

# Check if systemd-resolved is managing DNS
ls -la /etc/resolv.conf
# /etc/resolv.conf -> ../run/systemd/resolve/stub-resolv.conf

# Use resolvectl for systemd-resolved management
resolvectl status               # Show per-interface DNS configuration
resolvectl query google.com     # Resolve with detailed statistics
resolvectl flush-caches         # Flush DNS cache

# In Kubernetes pods — resolv.conf is injected by kubelet:
cat /etc/resolv.conf   # Inside a pod
# nameserver 10.96.0.10         ← CoreDNS ClusterIP
# search default.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5

# The ndots:5 in Kubernetes explained:
# Query: "database" (0 dots, less than 5)
# → tries: database.default.svc.cluster.local   (NXDOMAIN)
# → tries: database.svc.cluster.local           (NXDOMAIN)
# → tries: database.cluster.local               (NXDOMAIN)
# → tries: database                             (finally!)
#
# Problem for external FQDNs: "api.github.com" has 2 dots < 5
# → tries: api.github.com.default.svc.cluster.local (NXDOMAIN)
# → tries: api.github.com.svc.cluster.local (NXDOMAIN)
# → tries: api.github.com.cluster.local (NXDOMAIN)
```

```
# → tries: api.github.com (succeeds — but 3 wasted DNS queries first!)
# Fix: add trailing dot → api.github.com. (explicit FQDN)
```

🔷 `/etc/nsswitch.conf` — Name Service Switch

```
cat /etc/nsswitch.conf
# hosts: files dns myhostname

# This controls the ORDER and SOURCES for hostname lookups:
# 1. files     → check /etc/hosts first
# 2. dns       → query DNS servers from /etc/resolv.conf
# 3. myhostname → resolve the local machine's hostname via systemd

# Other important entries in nsswitch.conf:
# passwd:   files systemd   ← User account lookup: local files, then
systemd
# group:    files systemd   ← Group lookup
# shadow:   files            ← Password file (local only for security)

# LDAP/Active Directory integration adds:
# hosts: files dns ldap      ← Also query LDAP directory for hostnames

# ⚠ Critical: if you want /etc/hosts to override DNS (the default):
# hosts: files dns           ← files checked FIRST (standard)
# If accidentally changed to:
# hosts: dns files           ← DNS checked first, /etc/hosts is fallback!
```

🔷 DNS in Kubernetes — full detail

```
# Every pod gets a generated /etc/resolv.conf injected by kubelet:
# nameserver 10.96.0.10        ← CoreDNS service ClusterIP
# search default.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5

# DNS lookup order for service name "database" inside a pod:
# 1. database.default.svc.cluster.local   ← Same namespace (most specific)
# 2. database.svc.cluster.local
# 3. database.cluster.local
# 4. database                             ← External DNS

# Full FQDN for a service (always works, no search domain needed):
# <service>.<namespace>.svc.cluster.local
# e.g.: postgres.database.svc.cluster.local

# CoreDNS configuration (ConfigMap in kube-system)
kubectl get configmap coredns -n kube-system -o yaml
```

```
# Debug DNS from inside a pod
kubectl run dns-debug --image=busybox --rm -it -- nslookup kubernetes
kubectl run dns-debug --image=busybox --rm -it -- cat /etc/resolv.conf

# Check CoreDNS is healthy
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

🔷 Short crisp interview answer

> "DNS resolution order is controlled by `/etc/nsswitch.conf` — by default `files dns`, meaning `/etc/hosts` is checked first, then the DNS servers in `/etc/resolv.conf`. The `search` directive in resolv.conf appends domain suffixes to short names, and `ndots` controls when search domains are tried before going directly to external DNS. In Kubernetes this matters significantly — pods have `ndots:5` so external FQDNs with fewer than 5 dots get 3 wasted DNS queries against search domains first. I flush DNS cache with `resolvectl flush-caches` on systemd systems. Key debugging: `dig @<nameserver> <name>` to test a specific server."

⚠ Tricky gotchas

```
# GOTCHA 1: Editing /etc/resolv.conf directly on modern systems
# On Ubuntu with systemd-resolved, it's a symlink
# Direct edits get overwritten by NetworkManager on next event!
# Use: resolvectl  OR  edit /etc/systemd/resolved.conf

# GOTCHA 2: ndots in Kubernetes causing latency
# Service mesh and external API calls suffer from extra NXDOMAIN queries
# Fix: use fully qualified names with trailing dot: "api.github.com."
# Or set ndots:1 in pod's dnsConfig for external-heavy workloads

# GOTCHA 3: /etc/hosts changes don't affect running processes
# Some apps cache DNS/hosts at startup — restart needed to pick up changes

# GOTCHA 4: DNS negative caching
# NXDOMAIN responses are cached too (negative TTL from SOA record)
# Can't override a negative-cached NXDOMAIN by editing /etc/hosts
# Wait for TTL or: resolvectl flush-caches

# GOTCHA 5: Different DNS in containers vs host
# Container DNS (Docker/K8s) is separate from host DNS
# dig inside a container goes to CoreDNS/Docker DNS, not host's
/etc/resolv.conf
```

5.8 `ssh` Internals — Key Exchange, Tunnels, `~/.ssh/config`, `ProxyJump` 🟡

🔷 How SSH works internally

```
SSH Handshake:
Client                                Server
   |                                     |
   |──── TCP SYN ──────────────────────►| port 22
   |◄──── TCP SYN-ACK ─────────────────|
   |──── TCP ACK ──────────────────────►|
   |                                     |
   |◄──── Server version string ────────| "SSH-2.0-OpenSSH_8.9"
   |──── Client version string ────────►|
   |                                     |
   |◄──── Server host key (public) ─────| Client checks ~/.ssh/known_hosts
   |                                     |
   |═══════ KEY EXCHANGE (Diffie-Hellman) ═══════|
   |  Both sides derive the SAME session key        |
   |  without ever transmitting it!                 |
   |                                                |
   |  All traffic from here is encrypted            |
   |                                                |
   |──── Authentication request ──────────────────►|
   |     (publickey or password)                    |
   |◄──── Auth success ──────────────────────────|
   |                                                |
   |═══════════════ Encrypted shell session ═══════|
```

🔷 Key-based authentication

```
# Generate key pair
ssh-keygen -t ed25519 -C "your@email.com"         # Recommended (modern,
fast, secure)
ssh-keygen -t rsa -b 4096 -C "your@email.com"     # RSA fallback for older
systems

# Files created:
# ~/.ssh/id_ed25519       ← Private key (NEVER share — keep this secret!)
# ~/.ssh/id_ed25519.pub   ← Public key (safe to distribute to servers)

# Copy public key to remote server
ssh-copy-id user@server                           # Appends to
~/.ssh/authorized_keys
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server  # Specify key explicitly

# Manually add key (when ssh-copy-id is unavailable)
cat ~/.ssh/id_ed25519.pub | ssh user@server \
    'mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys
&& chmod 600 ~/.ssh/authorized_keys'

# Correct permissions — SSH refuses to use keys if too permissive!
```

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519           # Private key: owner read-only
chmod 644 ~/.ssh/id_ed25519.pub       # Public key: world-readable is fine
chmod 600 ~/.ssh/authorized_keys      # Authorized keys: owner read/write
only
chmod 600 ~/.ssh/config               # Config file: owner read/write only

# ssh-agent — avoid re-entering passphrase every time
eval $(ssh-agent)                     # Start agent
ssh-add ~/.ssh/id_ed25519             # Add key to agent (enter passphrase
once)
ssh-add -l                            # List keys currently in agent
ssh-add -D                            # Remove all keys from agent
```

🔷 `~/.ssh/config` — Your productivity multiplier

```
# ~/.ssh/config

# ── Global defaults ──────────────────────────────────────────────
Host *
    ServerAliveInterval 60          # Send keepalive every 60 seconds
    ServerAliveCountMax 3           # Drop connection after 3 missed
keepalives
    AddKeysToAgent yes              # Auto-add keys to ssh-agent
    IdentityFile ~/.ssh/id_ed25519  # Default key to use

# ── Named host alias ─────────────────────────────────────────────
Host prod-web
    HostName 10.0.0.5               # Actual IP or hostname
    User ubuntu
    Port 22
    IdentityFile ~/.ssh/prod_key

# ── Wildcard matching ────────────────────────────────────────────
Host staging-*
    User deploy
    IdentityFile ~/.ssh/staging_key
    StrictHostKeyChecking no        # ⚠ Only for ephemeral/auto-
provisioned instances

# ── Bastion/jump host ────────────────────────────────────────────
Host internal-db
    HostName 10.0.1.50              # Private IP — not directly reachable
    User postgres
    ProxyJump bastion               # Jump through bastion host first

Host bastion
    HostName bastion.example.com    # Public bastion IP/hostname
    User ec2-user
    IdentityFile ~/.ssh/bastion_key
```

```
# After this config, ONE command does everything:
# ssh internal-db   → connects via bastion automatically!
```

🔷 SSH tunnels — Port forwarding

```
# ── Local port forwarding (-L) ──────────────────────────────────────
# "Listen on LOCAL port, forward traffic through SSH tunnel to remote
destination"
# Syntax: -L [local_addr:]local_port:remote_host:remote_port

ssh -L 8080:localhost:80 user@server
# Now: curl http://localhost:8080 → tunnels through SSH → server:80

# Access internal database via bastion
ssh -L 5432:internal-db.corp:5432 user@bastion
# Now: psql -h localhost -p 5432  → tunnels to internal-db:5432

# -N: don't open a shell (tunnel only)
# -f: go to background after authentication
ssh -NfL 5432:internal-db.corp:5432 user@bastion

# ── Remote port forwarding (-R) ─────────────────────────────────────
# "Listen on REMOTE port, forward traffic back to LOCAL machine"
# Syntax: -R remote_port:local_host:local_port

ssh -R 9090:localhost:3000 user@server
# On the remote server: curl http://localhost:9090 → tunnels back to YOUR
machine:3000
# Use case: expose local dev server to a colleague temporarily

# ── Dynamic forwarding (-D) — SOCKS proxy ───────────────────────────
ssh -D 1080 user@server
# Configure browser/app to use SOCKS5 proxy at localhost:1080
# All traffic routes through the remote server
# Use case: browse internal systems as if on the server's network

# ── ProxyJump (-J) — multi-hop SSH ──────────────────────────────────
# Connect to internal server via bastion in one command
ssh -J ec2-user@bastion.example.com ubuntu@10.0.1.50

# Chain multiple jump hosts (SSH 7.3+)
ssh -J user@hop1,user@hop2 user@final-destination

# In ~/.ssh/config (cleaner and permanent):
# ProxyJump bastion
# Then just: ssh internal-db
```

🔷 SSH security hardening

```
# /etc/ssh/sshd_config — critical server hardening settings

PasswordAuthentication no       # Disable passwords — keys only
PermitRootLogin no              # Never allow direct root login
AllowUsers deploy ubuntu        # Whitelist specific users (deny all
others)
Port 2222                       # Non-standard port (reduces log noise)
MaxAuthTries 3                  # Reduce brute force window
LoginGraceTime 20               # Only 20 seconds to authenticate
ClientAliveInterval 300         # Send keepalive every 5 minutes
ClientAliveCountMax 2           # Drop after 2 missed keepalives
X11Forwarding no                # Disable X11 forwarding if not needed
AllowTcpForwarding no           # Disable port forwarding if not needed
Banner /etc/ssh/banner.txt      # Show legal warning banner at login

# CRITICAL: Test config before reloading (avoid lockout!)
sudo sshd -t                    # Syntax check — no output = no errors
sudo sshd -T | grep "permitroot"  # Check effective setting value

# Apply changes without killing existing sessions
sudo systemctl reload sshd      # ✅ Keeps existing sessions alive
sudo systemctl restart sshd     # ❌ Kills ALL existing sessions!

# Audit access
who                             # Currently logged-in users
w                               # Who + what they're doing
last | head -20                 # Recent successful logins
lastb | head -20                # Recent FAILED login attempts
```

🔷 Short crisp interview answer

> "SSH uses Diffie-Hellman key exchange to establish a shared session key without ever transmitting it — both sides independently compute the same secret. The server authenticates itself via its host key stored in `~/.ssh/known_hosts`, and the user authenticates via public key (server checks `~/.ssh/authorized_keys`). In production I always use `~/.ssh/config` with `ProxyJump` for bastion access — `ssh -J bastion internal-db` creates a direct encrypted tunnel through the jump host without exposing the agent or private key to the intermediate machine. For tunnels: `-L` pulls a remote service to a local port, `-R` exposes a local port on the remote server. Critical: always `reload` sshd — never `restart` which kills active sessions."

⚠️ Tricky gotchas

```
# GOTCHA 1: known_hosts mismatch warning
# @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
# @ WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED! @
```

```
# This is EITHER: a genuine MITM attack OR the server was
rebuilt/redeployed
# Fix (only if you KNOW it's the same trusted server):
ssh-keygen -R hostname              # Remove old key
ssh user@hostname                   # Re-connect and accept new key

# GOTCHA 2: ~/.ssh/ permissions must be exact
# SSH silently refuses to use private keys if permissions are too open!
chmod 700 ~/.ssh                    # Required
chmod 600 ~/.ssh/id_ed25519         # Required — not 644 or 666!

# GOTCHA 3: ProxyJump vs ProxyCommand
# ProxyJump  (SSH 7.3+): -J user@bastion  ← simpler, preferred
# ProxyCommand (older):  -o ProxyCommand='ssh -W %h:%p bastion'  ← legacy

# GOTCHA 4: Agent forwarding (-A) is LESS secure than ProxyJump
# -A forwards your SSH agent socket to the intermediate server
# A compromised intermediate server can use your agent to authenticate
elsewhere!
# ProxyJump creates a direct tunnel — agent never touches the intermediate
host
# ALWAYS prefer ProxyJump over agent forwarding

# GOTCHA 5: sshd reload vs restart
sudo systemctl reload sshd      # ✅ Reloads config, existing sessions
continue
sudo systemctl restart sshd     # ❌ KILLS all active sessions — use only
if required
```

5.9 TCP States — `SYN_SENT`, `TIME_WAIT`, `CLOSE_WAIT` 🔴

🔷 What it is in simple terms

Every TCP connection moves through a defined set of states from creation to destruction. Understanding these states is the difference between **diagnosing a real network problem** and chasing ghosts. Three states in particular — TIME_WAIT, CLOSE_WAIT, and SYN_RECV — are the ones that appear in production incidents and every senior SRE interview.

🔷 The full TCP state machine

```
                    ┌──────────────────┐
                    │      CLOSED      │
                    └──────────────────┘
                              │
                              │
            Server: bind() + listen()
            Client: connect() sends SYN
                              │
                              │
           ┌──────────────────┼──────────────────────────┐
           │ Server path      │                          │ Client path
```

```
                ▼                                       ▼
     ┌──────────────────┐◄─SYN──┐          ┌──────────────────┐
     │    SYN_RECV      │       │          │     SYN_SENT     │
     │ (got SYN,        │       │          │ (sent SYN,       │
     │  sent SYN-ACK,   │       │          │  waiting SYN-ACK)│
     │  waiting ACK)    │       │          └──────────────────┘
     └──────────────────┘                            │ got SYN-ACK
             │ got ACK                               │ sends ACK
             │                                       │
             └───────────────────────────────────────┘
                                ▼
             ┌──────────────────────────────────────┐
             │              ESTABLISHED             │
             │       (data transfer happens here)   │
             └──────────────────────────────────────┘
                                │
              ┌─────────────────────────────────────┐
              │ Active close                        │ Passive close
              │ (this side calls close() first)     │ (remote
called close() first)
              │ sends FIN                           │ receives FIN
              ▼                                     ▼
     ┌──────────────────┐                  ┌──────────────────┐
     │   FIN_WAIT_1     │                  │    CLOSE_WAIT    │
     │ (sent FIN,       │                  │ ← APP BUG if it  │
     │  waiting ACK)    │                  │   stays here!    │
     └──────────────────┘                  └──────────────────┘
             │ got ACK                               │ app calls
close()
             ▼                                       │ sends FIN
                                                     ▼
     ┌──────────────────┐                  ┌──────────────────┐
     │   FIN_WAIT_2     │                  │     LAST_ACK     │
     │ (our FIN acked,  │                  │ (sent FIN, waiting│
     │  waiting remote  │                  │  final ACK)      │
     │  FIN)            │                  └──────────────────┘
     └──────────────────┘                          │ got ACK
             │ got remote FIN                      ▼
             │ sends ACK                     ┌──────────┐
             ▼                               │ CLOSED   │
     ┌──────────────────┐                    │          │
     │    TIME_WAIT     │──── 2×MSL timeout ──────────►└──────────┘
     │ (waiting ~2 min) │     (~2 minutes)
     └──────────────────┘
```

🔷 TIME_WAIT — the most common state in production

```
# What it is:
# After the ACTIVE CLOSER sends the final ACK, it enters TIME_WAIT
# It stays here for 2×MSL (Maximum Segment Lifetime)
# MSL is typically 60 seconds → TIME_WAIT lasts ~2 minutes
```

```
# Why it exists — TWO critical reasons:
# 1. Ensure final ACK arrived (if lost, remote resends FIN and we can re-
ACK)
# 2. Absorb delayed packets from old connection (prevent them polluting
new one)

# Check TIME_WAIT count
ss -t state time-wait | wc -l
ss -tn state time-wait | head -20    # See which connections

# Is a large number a problem?
# TIME_WAIT is NORMAL and indicates healthy connection turnover
# It becomes a problem ONLY if you exhaust the ephemeral port range

# Check your ephemeral port range
cat /proc/sys/net/ipv4/ip_local_port_range
# 32768   60999   ← ~28000 available ports

# Each TIME_WAIT entry holds ONE port for ~2 minutes
# Under extreme load: 28000 ports ÷ 2 minutes = max ~233 new
connections/second
# This is why high-throughput microservices hit port exhaustion

# Fix options:
# Option 1: Widen port range
sudo sysctl net.ipv4.ip_local_port_range="1024 65535"

# Option 2: Allow reuse of TIME_WAIT ports for NEW connections
sudo sysctl net.ipv4.tcp_tw_reuse=1

# Option 3: Connection pooling in the application (best fix)
# Don't create/destroy connections per request — reuse them

# Option 4: HTTP keepalive (for HTTP services)
# Keep TCP connections alive across multiple HTTP requests

# ⚠ tcp_tw_recycle was REMOVED in Linux 4.12 — do NOT use it
# It caused failures behind NAT (broke clients sharing an IP)
```

🔷 CLOSE_WAIT — the dangerous one ⚠

```
# What it is:
# The REMOTE side sent FIN (called close())
# YOUR APPLICATION has received it BUT has not called close() yet
# Your app is stuck holding the socket open

# Why it's always an app bug:
# CLOSE_WAIT = "I know you want to close, but my app hasn't closed its
end"
```

```
# Every CLOSE_WAIT connection is a socket your application is leaking
# No sysctl setting can fix this — you MUST fix the application code

# Check for CLOSE_WAIT accumulation (the danger sign)
ss -tn state close-wait | wc -l
# If this number grows over time = connection leak = app bug

# Diagnose which process is leaking
sudo ss -tnp state close-wait
# Shows: CLOSE-WAIT  0  0  10.0.0.1:80  1.2.3.4:54321  users:
(("python3",pid=1234))

# Check how many file descriptors that process has open
sudo lsof -p 1234 | wc -l
# Growing fd count confirms the leak

# Root causes of CLOSE_WAIT leaks:
# - Not closing database connections after query
# - Exception swallowing before close() is called
# - try/catch that catches but doesn't re-throw, skipping the finally
block
# - Thread blocked waiting on I/O, holding connection open
# - Connection pool not returning connections to pool on error path
# - Missing timeout on blocking read

# Fix pattern in Python:
# ❌ Wrong
def query():
    conn = db.connect()
    result = conn.execute(sql)    # If this throws, conn never closed!
    return result

# ✅ Correct
def query():
    with db.connect() as conn:    # Context manager guarantees close()
        return conn.execute(sql)
```

🔷 SYN_RECV — potential attack indicator

```
# Normal behavior:
# SYN_RECV is extremely transient — milliseconds
# Server received SYN, sent SYN-ACK, waiting for client's ACK

# Under SYN flood attack:
# Attacker sends SYN with spoofed source IPs
# Server sends SYN-ACK to fake IPs — ACK never comes
# Server's SYN backlog fills up with SYN_RECV entries
# Legitimate connections get refused!

# Check SYN_RECV count
```

```
ss -tn state syn-recv | wc -l
# Normal: near 0
# Under attack: thousands

# Defense: SYN cookies (should be enabled by default)
sysctl net.ipv4.tcp_syncookies
# Should be: 1

# How SYN cookies work:
# Instead of storing state per half-open connection,
# the server encodes the state IN the ISN (Initial Sequence Number)
# When the legitimate ACK arrives, the server decodes the state
# Result: SYN flood doesn't exhaust server memory

# Increase SYN backlog for high-traffic servers
sudo sysctl net.ipv4.tcp_max_syn_backlog=65535
sudo sysctl net.core.somaxconn=65535    # Also increase listen() backlog
```

🔷 Key sysctl tuning parameters

```
# View all current TCP parameters
sysctl -a | grep net.ipv4.tcp | sort

# ── Most important tuning knobs ──────────────────────────────────

# How long to keep FIN_WAIT_2 state (default 60s)
sysctl net.ipv4.tcp_fin_timeout

# TCP keepalive settings (when to detect dead connections)
sysctl net.ipv4.tcp_keepalive_time      # First probe after N seconds
(default 7200 = 2h!)
sysctl net.ipv4.tcp_keepalive_intvl     # Interval between probes (default
75s)
sysctl net.ipv4.tcp_keepalive_probes    # Number of probes before giving
up (default 9)

# Port range for outbound connections
sysctl net.ipv4.ip_local_port_range     # default: 32768-60999

# Listen backlog limits
sysctl net.core.somaxconn               # Max listen backlog (default 128
- too low!)
sysctl net.ipv4.tcp_max_syn_backlog     # Max half-open connections

# TIME_WAIT settings
sysctl net.ipv4.tcp_tw_reuse            # Allow reuse of TIME_WAIT ports
(default 0)

# ── Production tuning for high-traffic servers ──────────────────
sudo sysctl net.core.somaxconn=65535
```

```
sudo sysctl net.ipv4.tcp_max_syn_backlog=65535
sudo sysctl net.ipv4.ip_local_port_range="1024 65535"
sudo sysctl net.ipv4.tcp_tw_reuse=1
sudo sysctl net.ipv4.tcp_fin_timeout=15
sudo sysctl net.ipv4.tcp_keepalive_time=600
sudo sysctl net.ipv4.tcp_keepalive_intvl=30
sudo sysctl net.ipv4.tcp_keepalive_probes=5

# Make permanent (persist across reboots)
cat >> /etc/sysctl.d/99-tcp-tuning.conf << 'EOF'
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
EOF
sudo sysctl -p /etc/sysctl.d/99-tcp-tuning.conf
```

🔷 The diagnostic workflow for TCP state issues

```
# Step 1: Count states to understand the pattern
ss -t | awk 'NR>1 {print $1}' | sort | uniq -c | sort -rn
# Output might be:
# 3421  ESTAB
#  842  TIME-WAIT
#   17  CLOSE-WAIT    ← Watch this one!
#    2  LISTEN

# Step 2: If CLOSE_WAIT is growing — find the leaking process
sudo ss -tnp state close-wait | awk '{print $NF}' | sort | uniq -c | sort
-rn

# Step 3: If TIME_WAIT is causing port exhaustion
# Check if ephemeral ports are exhausted
ss -s | grep "TCP:"
cat /proc/sys/net/ipv4/ip_local_port_range
# Compare port range size against TIME_WAIT count

# Step 4: If SYN_RECV is high — check for attack
ss -tn state syn-recv | wc -l
dmesg | grep -i "syn flood"
netstat -s | grep "SYNs to LISTEN"

# Step 5: Application-level validation
# Even if TCP is fine, check app-level health
curl -o /dev/null -s -w "%{http_code}" http://localhost/health
```

🔷 Short crisp interview answer

> "TCP has 11 states but three matter most in production. `TIME_WAIT` is normal — the active closer waits 2×MSL (~2 minutes) after the final ACK to absorb delayed packets and handle lost ACKs. High TIME_WAIT counts are expected under load; the fix for exhaustion is `tcp_tw_reuse=1` or connection pooling. `CLOSE_WAIT` is the dangerous one — the remote side closed but your application hasn't called `close()` yet. Growing CLOSE_WAIT is always an application bug — a connection leak — and no sysctl tuning can fix it. `SYN_RECV` in high counts signals a SYN flood; `tcp_syncookies=1` (default on modern kernels) defends against it by encoding connection state in the sequence number. I diagnose with `ss -t | awk '{print $1}' | sort | uniq -c`."

⚠ Tricky gotchas

```
# GOTCHA 1: CLOSE_WAIT is ALWAYS an application bug — not a network issue
# Never try to fix CLOSE_WAIT with sysctl — it's wasted effort
# Fix the application: ensure close() is called on all code paths

# GOTCHA 2: TIME_WAIT accumulates on the CLIENT side (active closer)
# In microservice architectures, your service IS the client for downstream
calls
# Your service accumulates TIME_WAIT, not the downstream server
# Watch the CALLER's port range, not just the server

# GOTCHA 3: Ephemeral port exhaustion looks like connection refused
# Symptom: "Cannot assign requested address" error
# Cause: all ports in ip_local_port_range are in TIME_WAIT
# Diagnosis: ss -s | grep TIME-WAIT count vs port range size

# GOTCHA 4: tcp_tw_recycle was REMOVED in Linux 4.12
# Old blog posts and Stack Overflow answers recommend it — DON'T use it
# It caused connection failures for clients behind NAT (multiple clients =
one IP)
# Kernel removed it entirely

# GOTCHA 5: somaxconn=128 is almost always too low
# Default listen backlog is 128 connections
# Under high load, excess connections get silently dropped
# Nginx/Apache often set higher limits in their own config — match the OS
limit
# Check: ss -tlnp and look at the Send-Q column on LISTEN sockets
```

5.10 Network Namespaces — Container Isolation 🔴

🔷 What it is in simple terms

A network namespace is a **completely isolated copy of the entire network stack** — its own interfaces, IP addresses, routing table, iptables rules, and port space. Two processes in different network namespaces can both listen on port 80 without conflict. This is the **fundamental kernel technology** behind Docker and Kubernetes networking.

🔷 How it works — the big picture

```
HOST NETWORK NAMESPACE
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  eth0 (203.0.113.5/24)    lo (127.0.0.1)                             │
│  routing table: default via 203.0.113.1                             │
│  iptables: MASQUERADE, DNAT rules                                    │
│                                                                      │
│  docker0 bridge (172.17.0.1/16) ──────────────────────────┐          │
│                                                           │          │
│   ┌──────────────────────────────────┐   veth pair       │          │
│   │  CONTAINER NAMESPACE 1           │   ┌─────────┐     │          │
│   │  eth0 (172.17.0.2/16)            │   │ veth0 ◄─────────┘          │
│   │  lo (127.0.0.1)                  │   │ veth1 │  (in ns)          │
│   │  route: default via 172.17.0.1   │   └─────────┘                  │
│   │  its own iptables                │                                │
│   └──────────────────────────────────┘                                │
│                                                                       │
│   ┌──────────────────────────────────┐   veth pair                   │
│   │  CONTAINER NAMESPACE 2           │                               │
│   │  eth0 (172.17.0.3/16)            │   ┌─────────┐                 │
│   │  lo (127.0.0.1)                  │   │ veth2 ◄──────────────────┘│
│   │  its own routing + iptables      │   └─────────┘                 │
│   └──────────────────────────────────┘                               │
└──────────────────────────────────────────────────────────────────────┘
```

🔷 Creating and managing network namespaces

```
# Create a new network namespace
sudo ip netns add myns

# List all namespaces
ip netns list
# myns

# Run a command INSIDE the namespace
sudo ip netns exec myns ip addr
# 1: lo: <LOOPBACK> mtu 65536 state DOWN
# Only loopback — completely isolated from the host!

sudo ip netns exec myns ping 8.8.8.8
# ping: connect: Network is unreachable  ← no routes configured yet

# Get a shell inside the namespace
sudo ip netns exec myns bash

# Delete a namespace
```

```
sudo ip netns del myns

# ── Connect namespace to the host with a veth pair ──────────────────

# Create a veth pair (two ends of a virtual ethernet cable)
sudo ip link add veth-host type veth peer name veth-ns

# Move one end into the namespace
sudo ip netns add myns
sudo ip link set veth-ns netns myns

# Configure the HOST end
sudo ip addr add 192.168.100.1/24 dev veth-host
sudo ip link set veth-host up

# Configure the NAMESPACE end (from inside the namespace)
sudo ip netns exec myns ip addr add 192.168.100.2/24 dev veth-ns
sudo ip netns exec myns ip link set veth-ns up
sudo ip netns exec myns ip link set lo up
sudo ip netns exec myns ip route add default via 192.168.100.1

# Now the namespace can ping the host:
sudo ip netns exec myns ping 192.168.100.1   # Works!

# Enable NAT so namespace can reach the internet
sudo sysctl net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING \
    -s 192.168.100.0/24 -o eth0 -j MASQUERADE

# Now the namespace can reach the internet:
sudo ip netns exec myns ping 8.8.8.8   # Works through NAT!
```

🔷 Viewing Docker/container namespaces

```
# Docker creates namespaces but doesn't register them with ip netns
# (they don't appear in ip netns list)

# Method 1: Find via container's PID
docker inspect mycontainer --format '{{.State.Pid}}'
# 12345

# Each process has its network namespace at:
ls -la /proc/12345/ns/net
# lrwxrwxrwx  /proc/12345/ns/net -> net:[4026532345]

# Method 2: Make it visible to ip netns
sudo mkdir -p /var/run/netns
sudo ln -s /proc/12345/ns/net /var/run/netns/mycontainer
ip netns list   # Now shows mycontainer!
```

```
# Now run ip commands in the container's namespace from the HOST
sudo ip netns exec mycontainer ip addr       # See container's interfaces
sudo ip netns exec mycontainer ip route      # See container's routing
table
sudo ip netns exec mycontainer ss -tlnp      # See container's listening
ports

# Alternative: enter namespace with nsenter
sudo nsenter -t 12345 -n ip addr
sudo nsenter --target 12345 --net ip route
```

🔷 How Docker uses namespaces end-to-end

```
# When you run: docker run -p 8080:80 nginx
# Docker does ALL of this automatically:

# 1. Creates new network namespace for the container
sudo ip netns add docker-<hash>

# 2. Creates veth pair
sudo ip link add veth12345 type veth peer name eth0

# 3. Moves one end into container namespace as eth0
sudo ip link set eth0 netns docker-<hash>

# 4. Connects host end to docker0 bridge
sudo ip link set veth12345 master docker0
sudo ip link set veth12345 up

# 5. Assigns IP inside the container
sudo ip netns exec docker-<hash> ip addr add 172.17.0.2/16 dev eth0
sudo ip netns exec docker-<hash> ip route add default via 172.17.0.1

# 6. Sets up iptables for internet access (MASQUERADE/SNAT)
sudo iptables -t nat -A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j
MASQUERADE

# 7. Sets up port mapping with DNAT
# docker run -p 8080:80 creates:
sudo iptables -t nat -A DOCKER -p tcp --dport 8080 \
    -j DNAT --to-destination 172.17.0.2:80

# Inspect the Docker iptables rules
sudo iptables -t nat -L DOCKER -n -v
sudo iptables -L DOCKER -n -v
```

🔷 Kubernetes networking overview

```
# Kubernetes networking adds another layer:
#
# Pod networking (per-node):
# - Each Pod gets its own network namespace
# - CNI plugin (Calico, Flannel, Cilium) creates the veth pair
# - All Pods on a node connect to a bridge or directly via routing
#
# Cross-node Pod communication:
# - CNI assigns each node a Pod CIDR (e.g., node1: 10.244.1.0/24)
# - Routes between nodes: "To reach 10.244.2.0/24, go to node2's IP"
# - Implemented via: host routes, VXLAN overlay, BGP (Calico), eBPF
(Cilium)
#
# Service networking (kube-proxy):
# - ClusterIP is a VIRTUAL IP — no real interface has this IP
# - kube-proxy programs iptables DNAT rules per Service
# - Traffic to ClusterIP:port → DNAT → one of the Pod endpoints
# - Modern: Cilium replaces kube-proxy with eBPF (no iptables, faster)

# Debug Pod network from the host
kubectl get pod my-pod -o wide              # Get pod IP and node
kubectl exec my-pod -- ip addr              # Interfaces inside pod
kubectl exec my-pod -- ip route             # Routes inside pod
kubectl exec my-pod -- cat /etc/resolv.conf # DNS config inside pod

# Debug from the node the pod runs on
# Find the veth for the pod
ip link show | grep -A2 "veth"
# Look for the veth pair whose peer is in the pod's namespace
```

🔷 Short crisp interview answer

> "Network namespaces give each container a completely isolated network stack — its own interfaces, routing table, iptables rules, and port space. Docker creates a namespace per container, connects it to the host via a `veth` pair (virtual ethernet cable), attaches the host end to the `docker0` bridge, and gives the container an IP from 172.17.0.0/16. Internet access works via iptables MASQUERADE (SNAT). Port mapping (`-p 8080:80`) is an iptables DNAT rule — traffic arriving on the host's port 8080 gets rewritten to the container's IP:80 by the PREROUTING chain. I debug container networking from the host using `ip netns exec` or `nsenter -t <pid> -n`."

⚠ Tricky gotchas

```
# GOTCHA 1: Docker namespaces don't appear in ip netns list
# Docker stores namespace refs in /proc/<pid>/ns/net, not /var/run/netns/
# Fix: ln -s /proc/<pid>/ns/net /var/run/netns/name

# GOTCHA 2: ip_forward must be enabled for routing between namespaces
sysctl net.ipv4.ip_forward   # Check — must be 1
```

```
# Docker enables this automatically — but if you reset sysctl, Docker
breaks!

# GOTCHA 3: Port binding in namespace is independent
# Container can bind port 80 even if host is already using port 80
# They're in different namespaces — no conflict
# The iptables DNAT rule is what maps host port → container port

# GOTCHA 4: Loopback is namespace-local
# 127.0.0.1 inside a container refers to the CONTAINER, not the host
# To reach the host from a container: use the docker0 bridge IP
(172.17.0.1)
# Or use host.docker.internal (Docker Desktop) or --network=host

# GOTCHA 5: Kubernetes Pods share network namespace within a Pod
# All containers IN THE SAME POD share one network namespace
# They communicate via localhost — no pod IP needed
# They CANNOT both bind the same port
```

5.11 eBPF Basics — Modern Linux Networking Observability 🔴

🔷 What it is in simple terms

eBPF (**extended Berkeley Packet Filter**) lets you run **sandboxed programs inside the Linux kernel** without modifying kernel source code, loading kernel modules, or rebooting. It's the most significant Linux infrastructure innovation of the past decade — used by Netflix, Cloudflare, Facebook, Google, and every major cloud provider.

🔷 Why it's revolutionary — the problem it solves

```
TRADITIONAL APPROACH — adding kernel observability:
  Need to trace what TCP connections are being made
      │
      ▼
  Option 1: Add code to kernel → recompile kernel → reboot server
            ← Weeks of work, full downtime, risky
      │
      ▼
  Option 2: Use tcpdump → copies ALL matching packets to userspace
            ← High CPU overhead at scale, misses kernel internals
      │
      ▼
  Option 3: Write a kernel module → loaded at runtime
            ← Can crash kernel, security risk, complex development

eBPF APPROACH:
  Write eBPF program → Load via bpf() syscall → Kernel verifier checks it
      │
      ▼
```

```
Verifier confirms: no infinite loops, no bad memory access, terminates
    │
    ▼
JIT compiled to native machine code → runs at kernel speed
    │
    ▼
Data aggregated IN KERNEL → only summaries sent to userspace
    │
    ▼
Near-zero overhead. No reboot. No kernel modification. Safe.
```

🔷 How eBPF works internally

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              LINUX KERNEL                                 │
│                                                                           │
│  HOOK POINTS — where eBPF programs attach:                                │
│                                                                           │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐         │
│   │  kprobes │   │tracepoints│   │    XDP   │   │ socket filter │         │
│   │(any kernel│   │(static   │   │(NIC driver│   │(per-socket   │         │
│   │ function) │   │ markers) │   │ level!)  │   │ filtering)   │         │
│   └──────────┘   └──────────┘   └──────────┘   └──────────────┘         │
│        │              │              │                │                   │
│        └──────────────┴──────────────┴────────────────┘                   │
│                                  │                                        │
│                                  ▼                                        │
│                        ┌──────────────────────┐                           │
│                        │    eBPF VERIFIER     │                           │
│                        │  Checks safety:      │                           │
│                        │  • No infinite loops │                           │
│                        │  • No bad ptr access │                           │
│                        │  • Program terminates│                           │
│                        │  • Stack bounds OK   │                           │
│                        └──────────────────────┘                           │
│                                  │ Verified                               │
│                                  ▼                                        │
│                        ┌──────────────────────┐                           │
│                        │    JIT COMPILER      │                           │
│                        │  eBPF bytecode →     │                           │
│                        │  native x86/ARM code │                           │
│                        └──────────────────────┘                           │
│                                  │                                        │
│                                  ▼                                        │
│                        Runs at kernel speed                               │
│                                  │                                        │
│                        ┌─────────▼────────────┐                           │
│                        │       eBPF MAPS      │                           │
│                        │  (shared memory:     │                           │
│                        │   kernel ↔ userspace)│                           │
```

```
│                          │  Hash maps, arrays,    │                  │
│                          │  ring buffers, etc.    │                  │
│                          └────────────────────────┘                  │
└──────────────────────────────────────────────────────────────────────┘
                                        │
                                        │
                              Userspace tools read maps
                              (bpftrace, bcc tools, etc.)
```

🔷 bcc tools — the networking toolkit

```
# Install bcc tools
sudo apt install bpfcc-tools linux-headers-$(uname -r)
# OR
pip3 install bcc

# ── tcpconnect — trace all new outbound TCP connections ────────────────
sudo tcpconnect-bpfcc
# PID    COMM         IP  SADDR           DADDR           DPORT
# 1234   curl         4   10.0.0.5        93.184.216.34   443
# 5678   python3      4   10.0.0.5        10.0.1.50       5432
# Catches connections the instant they're made — zero polling overhead

# ── tcpaccept — trace all accepted inbound connections ─────────────────
sudo tcpaccept-bpfcc
# PID    COMM         IP  RADDR           RPORT  LADDR           LPORT
# 910    nginx        4   1.2.3.4         54321  10.0.0.5        80

# ── tcplife — full connection lifecycle with duration and bytes ────────
sudo tcplife-bpfcc
# PID   COMM    LADDR      LPORT  RADDR       RPORT  TX_KB  RX_KB  MS
# 123   nginx   10.0.0.5   80     1.2.3.4     54321  0.5    12.3   145.2
# 456   curl    10.0.0.5   43210  8.8.8.8     443    1.2    0.1    89.4
# Shows total transfer + duration per connection — great for latency
analysis

# ── tcpretrans — trace TCP retransmissions (network quality) ───────────
sudo tcpretrans-bpfcc
# Shows each retransmission with: PID, address, port, TCP state
# Growing retransmit count = packet loss = network quality problem

# ── tcptop — top-like view of TCP bandwidth by process ─────────────────
sudo tcptop-bpfcc
# PID    COMM         LADDR:LPORT     RADDR:RPORT     RX_KB  TX_KB
# 910    nginx        10.0.0.5:80     1.2.3.4:54321   1024   256

# ── execsnoop — trace all process executions ───────────────────────────
sudo execsnoop-bpfcc
# Useful for: detecting unexpected processes spawned by your app

# ── opensnoop — trace all file opens ───────────────────────────────────
```

```
sudo opensnoop-bpfcc -p 1234    # Track specific PID
# Shows every file your process opens — great for config debugging
```

🔷 `bpftrace` — DTrace-like one-liners

```
# Install
sudo apt install bpftrace

# List available tracepoints and probes
sudo bpftrace -l 'tracepoint:net:*'
sudo bpftrace -l 'kprobe:tcp_*'

# ── Networking one-liners ──────────────────────────────────────

# Count TCP connections per destination port
sudo bpftrace -e '
kprobe:tcp_connect {
    @[((struct sock *)arg0)->__sk_common.skc_dport] = count()
}
interval:s:10 {
    print(@); clear(@)
}'

# Trace new connections with process name
sudo bpftrace -e '
tracepoint:sock:inet_sock_set_state
/args->newstate == 1/ {
    printf("%s → %s:%d\n", comm,
        ntop(args->daddr), args->dport)
}'

# Measure DNS query latency (UDP send/receive)
sudo bpftrace -e '
kprobe:udp_sendmsg { @start[tid] = nsecs; }
kretprobe:udp_recvmsg /@start[tid]/ {
    @latency_us = hist((nsecs - @start[tid]) / 1000);
    delete(@start[tid]);
}'

# Count packets per interface
sudo bpftrace -e '
tracepoint:net:netif_receive_skb {
    @[str(args->name)] = count()
}'

# Show large TCP sends (potential bandwidth hogs)
sudo bpftrace -e '
kprobe:tcp_sendmsg {
    if (arg2 > 65536) {
        printf("Large send: PID=%d comm=%s size=%d\n", pid, comm, arg2)
```

```
    }
}'
```

🔷 XDP — eXpress Data Path

```
# XDP = eBPF programs attached at the NIC DRIVER level
# Packets are processed BEFORE they enter the kernel network stack
# This means ZERO kernel overhead for dropped/redirected packets

# Performance comparison:
# iptables DROP:    ~3 Mpps (million packets per second)
# tc/BPF DROP:     ~10 Mpps
# XDP DROP:        ~25 Mpps on commodity NIC
# XDP with DPDK:   >100 Mpps

# XDP actions:
# XDP_DROP    → drop packet at NIC level (DDoS mitigation)
# XDP_PASS    → pass packet to normal kernel stack
# XDP_TX      → retransmit packet back out the same interface
# XDP_REDIRECT → redirect to another interface or CPU

# Real-world XDP deployments:
# Cloudflare: XDP for DDoS mitigation — drops attack traffic at line rate
# Facebook: Katran load balancer — XDP-based L4 load balancing at scale
# Cilium: XDP for Kubernetes NodePort service acceleration

# Example XDP program concept (C code compiled to eBPF bytecode):
# SEC("xdp")
# int xdp_drop_icmp(struct xdp_md *ctx) {
#     void *data = (void *)(long)ctx->data;
#     void *data_end = (void *)(long)ctx->data_end;
#     struct ethhdr *eth = data;
#     if ((void *)(eth + 1) > data_end) return XDP_PASS;
#     if (eth->h_proto == htons(ETH_P_IP)) {
#         struct iphdr *ip = (void *)(eth + 1);
#         if ((void *)(ip + 1) > data_end) return XDP_PASS;
#         if (ip->protocol == IPPROTO_ICMP) return XDP_DROP;
#     }
#     return XDP_PASS;
# }
```

🔷 Cilium — eBPF-native Kubernetes networking

```
# Cilium replaces kube-proxy (iptables) entirely with eBPF
# Traditional kube-proxy: O(n) iptables rules per Service — degrades at
scale
# Cilium eBPF: O(1) hash map lookup — scales to 100k+ services
```

```
# What Cilium does with eBPF:
# 1. Service load balancing (replaces kube-proxy iptables DNAT rules)
# 2. NetworkPolicy enforcement (L3/L4/L7 — including HTTP path-based
policies)
# 3. Observability via Hubble (no sidecar proxies needed)
# 4. mTLS between pods (no Istio/Envoy sidecar required)
# 5. Bandwidth management
# 6. Transparent encryption (WireGuard)

# Check Cilium status
cilium status
cilium connectivity test

# Hubble — eBPF-powered observability for Kubernetes
hubble observe                          # Live network flow visibility
hubble observe --pod frontend           # Flows involving a specific pod
hubble observe --verdict DROPPED        # Show only dropped connections
hubble observe --protocol http          # HTTP flows only

# Hubble UI — visual network traffic map in browser
cilium hubble ui
```

🔷 Why eBPF matters for SRE/DevOps roles

```
The 5 questions eBPF answers that previously required deep kernel work:

1. "Which process made this outbound connection to the database?"
   → tcpconnect-bpfcc: shows PID + process name + remote IP + port
instantly

2. "What is causing TCP retransmissions between service A and B?"
   → tcpretrans-bpfcc: shows each retransmit with full 5-tuple + state

3. "What is the latency breakdown inside the kernel for this syscall?"
   → bpftrace one-liner: histogram of syscall latency with zero app
changes

4. "Which container is using the most bandwidth right now?"
   → tcptop-bpfcc: per-process, per-connection bandwidth in real time

5. "We're under DDoS — how do we drop attack traffic at line rate?"
   → XDP program: attach to NIC, drop matching packets before kernel sees
them

Previously answering these required:
- Kernel instrumentation (modify + recompile + reboot)
- Sampling (miss fast events)
- High-overhead userspace capture
- Custom kernel modules (risky, version-specific)
```

```
With eBPF: minutes to attach, near-zero overhead, no reboots, safe
```

🔷 Short crisp interview answer

> "eBPF lets you run verified, JIT-compiled programs inside the Linux kernel at hook points — network ingress/egress, syscalls, function entry/exit — without kernel modifications or reboots. The verifier ensures programs are safe: no infinite loops, no bad memory accesses, always terminates. Data is aggregated in kernel via eBPF maps and only summaries reach userspace — making it near-zero overhead at production scale. For networking I use `tcpconnect` to trace every new connection with process name, `tcplife` for connection duration and bytes, and `tcpretrans` for detecting packet loss. XDP enables DDoS mitigation at 25+ million packets per second by dropping packets at the NIC driver before the kernel stack touches them. In Kubernetes, Cilium uses eBPF to replace iptables entirely — O(1) hash lookups vs O(n) iptables rules, scaling to hundreds of thousands of services."

⚠️ Tricky gotchas

```
# GOTCHA 1: eBPF requires a relatively modern kernel
# Most features: Linux 4.9+ (2016)
# Full feature set: Linux 5.x+
# Check: uname -r
# bpftrace: requires Linux 4.9+
# XDP: requires kernel + NIC driver support

# GOTCHA 2: eBPF requires root or CAP_BPF + CAP_PERFMON capabilities
sudo bpftrace ...     # Works
bpftrace ...          # Permission denied in most configs

# GOTCHA 3: bcc tool names vary by distribution
# Ubuntu: tcpconnect-bpfcc, tcpaccept-bpfcc, tcptop-bpfcc
# Fedora/RHEL: tcpconnect, tcpaccept, tcptop (no suffix)
# Check: ls /usr/sbin/tcp*

# GOTCHA 4: eBPF programs are per-kernel-version
# Compiled eBPF bytecode is not portable across kernel versions
# CO-RE (Compile Once Run Everywhere) solves this in modern kernels
# libbpf + BTF enables portable eBPF programs

# GOTCHA 5: eBPF observability adds overhead (small but non-zero)
# Each probe fire = small overhead
# High-frequency probes (e.g., every packet) can add up at 10Gbps
# Use sampling or aggregate in-kernel to minimize overhead
```

🏆 Category 5 — Complete Mental Model

```
NETWORKING TROUBLESHOOTING DECISION TREE
═══════════════════════════════════════════

"Can't reach service X"
    │
    ├─ 1. Is the HOST reachable at Layer 3?
    │       ping -c 3 <host>
    │       └─ No → traceroute -n <host>  (find where it breaks)
    │              Remember: * * * does NOT mean broken
    │
    ├─ 2. Is the PORT open and listening?
    │       ss -tlnp | grep :<PORT>          (on the server)
    │       nc -zv <host> <PORT>             (from the client)
    │
    ├─ 3. Is DNS resolving correctly?
    │       dig +short <hostname>            (uses configured resolver)
    │       dig @8.8.8.8 +short <hostname>  (bypass local resolver)
    │
    ├─ 4. Is a firewall blocking it?
    │       sudo iptables -L -n -v | grep <PORT>
    │       sudo tcpdump -nn 'port <PORT>'   (do packets arrive at all?)
    │
    ├─ 5. Is the route correct?
    │       ip route get <TARGET_IP>
    │       ip route show
    │
    ├─ 6. What do the packets look like on the wire?
    │       sudo tcpdump -nn -i eth0 'host X and port Y'
    │       sudo tcpdump -w /tmp/cap.pcap ... (save for Wireshark)
    │
    └─ 7. Advanced: TCP state / app-level diagnosis
            ss -t | awk '{print $1}' | sort | uniq -c
            sudo ss -tnp state close-wait  (connection leaks?)
            sudo tcpconnect-bpfcc          (trace live connections)

TOOL SELECTION GUIDE
────────────────────────────────────────
Is host alive?                → ping
Where does path break?        → traceroute -n  or  mtr
DNS resolution issue?         → dig @<server> <name> +short
What's listening on ports?    → ss -tulnp
Raw packet capture?           → tcpdump -nn -w file.pcap
Firewall rules inspection?    → iptables -L -n -v
Which route would be taken?   → ip route get <IP>
HTTP/API debugging?           → curl -sSfv  or  curl -w timing
TCP state analysis?           → ss -t state <state>
Container networking?         → ip netns exec + veth pair inspection
Production tracing?           → bcc tools (tcpconnect, tcplife,
tcpretrans)
DDoS mitigation at scale?     → XDP eBPF program
Kubernetes observability?     → Hubble (Cilium) + bpftrace
```

⚠ Master Gotcha List

#	Gotcha	The Truth
1	`ping` failure = host is down	ICMP is blocked on most cloud hosts; test with `nc -zv` or `curl`
2	`traceroute * * *` = broken path	Routers often drop ICMP TTL-exceeded; path usually continues fine
3	`curl` exits 0 even on 404/500	Always use `-f` flag in scripts to get non-zero exit on HTTP errors
4	High TIME_WAIT count = problem	TIME_WAIT is NORMAL; CLOSE_WAIT is the bug indicator
5	CLOSE_WAIT fixable by sysctl	It's ALWAYS an application bug — fix the code, not the kernel
6	iptables rules survive reboot	They DON'T — must use `iptables-save` / `iptables-restore`
7	First iptables rule match wins	Order matters critically — use `--line-numbers` to verify
8	`netstat` is the right tool	Deprecated; use `ss` which is faster (reads kernel directly)
9	`/etc/resolv.conf` is editable	Often managed by systemd-resolved/NetworkManager — edits get overwritten
10	SSH agent forwarding is safe	ProxyJump is safer — agent never touches the jump host
11	`tcpdump` shows HTTPS content	HTTPS is encrypted — only TCP metadata visible, not payload
12	`ip link UP` = interface connected	Need `LOWER_UP` flag too — that confirms physical link is active
13	`tcp_tw_recycle` fixes TIME_WAIT	Removed in Linux 4.12 — breaks NAT; DO NOT USE
14	Container DNS = host DNS	Container gets its own `/etc/resolv.conf` injected (CoreDNS in K8s)
15	eBPF has high overhead	Data aggregates in-kernel; userspace gets summaries — near-zero overhead

🔥 Top Interview Questions

Q1: What's the difference between `ping` failing and a service being down?

> `ping` uses ICMP which is frequently blocked by cloud security groups, host-based firewalls, and network ACLs. A service can be fully operational serving HTTP/HTTPS traffic while ping fails. I verify reachability at the application layer using `nc -zv host port` to test TCP connectivity, or `curl -sSf http://host/health` for HTTP services. Only when both ping AND TCP connection fail do I conclude the host is unreachable.

Q2: You see thousands of TIME_WAIT connections on your server — is this a problem?

> TIME_WAIT is normal TCP behavior — after the active closer sends the final ACK, it waits 2×MSL (~2 minutes) to absorb delayed packets and ensure the final ACK was received. High TIME_WAIT counts indicate healthy connection turnover. It only becomes a problem if you exhaust the ephemeral port range. I check: `cat /proc/sys/net/ipv4/ip_local_port_range` (typically 28,000 ports) vs TIME_WAIT count. Fixes: `tcp_tw_reuse=1` allows reusing TIME_WAIT ports for new outbound connections, or better — implement connection pooling in the application.

Q3: You see CLOSE_WAIT connections growing over time — what does this mean?

> CLOSE_WAIT means the remote side has sent FIN (called close) but the local application has not called close yet. Growing CLOSE_WAIT count is always an application-level connection leak — a code path that receives data then fails to close the connection. Common causes: missing close in error paths, exceptions caught before the finally block, blocked threads holding connections. I diagnose with `sudo ss -tnp state close-wait` to find the process, then `lsof -p <pid> | wc -l` to confirm fd count is growing. No sysctl can fix this — the code must be fixed.

Q4: How does a Docker container get network connectivity?

> Docker creates a network namespace per container, creates a veth pair (virtual ethernet cable), puts one end in the container namespace as `eth0` and connects the other end to the `docker0` bridge in the host namespace. The container gets an IP from 172.17.0.0/16. Internet access works via iptables MASQUERADE — container packets are SNAT'd to the host's IP before leaving. Port mapping (`-p 8080:80`) is an iptables DNAT rule in the PREROUTING chain that rewrites the destination to the container's IP:80.

Q5: Explain the DNS resolution chain inside a Kubernetes pod.

> Every pod has `/etc/resolv.conf` injected by kubelet pointing to the CoreDNS ClusterIP with `search default.svc.cluster.local svc.cluster.local cluster.local` and `ndots:5`. For a query like "database", CoreDNS first tries `database.default.svc.cluster.local` — which resolves if there's a Service named "database" in the same namespace. For external FQDNs with fewer than 5 dots (like `api.github.com`), it wastefully tries all three search domains first before falling back to external DNS — adding latency. Fix by using FQDNs with trailing dot: `api.github.com.`

Q6: How does SSH key authentication work without transmitting the private key?

> During authentication, the server encrypts a random challenge with the client's public key. Only the holder of the private key can decrypt it. The client decrypts the challenge and sends back an HMAC of the decrypted value combined with the session ID — this proves possession of the private key without ever transmitting it. The session encryption key itself is negotiated separately via Diffie-Hellman, where both sides independently compute the same shared secret without transmitting it either.

Q7: What is eBPF and why is it significant for production observability?

> eBPF lets you run verified, JIT-compiled programs at kernel hook points — network events, syscalls, function calls — without kernel modifications or reboots. The verifier ensures safety (no infinite loops, bounded memory access). The key performance insight is that data aggregation happens in-kernel via eBPF maps — only summaries reach userspace, unlike traditional tools that copy raw data. For networking: `tcpconnect` traces every new TCP connection with process name in real time, `tcpretrans` shows packet loss, `tcplife` shows connection duration. XDP programs at the NIC driver level can drop DDoS traffic at 25+ million packets per second. In Kubernetes, Cilium uses eBPF to replace iptables-based kube-proxy entirely, giving O(1) service lookups instead of O(n) iptables rule chains.

Q8: What is a SYN flood and how does Linux defend against it?

> In a SYN flood, an attacker sends SYN packets with spoofed source IPs. The server sends SYN-ACK to fake IPs that never respond, filling the SYN backlog with SYN_RECV entries until no legitimate connections can be accepted. Linux defends with SYN cookies — instead of storing state per half-open connection, the server encodes the connection parameters inside the ISN (Initial Sequence Number). When a legitimate ACK arrives, the server decodes the state from the sequence number. The backlog never fills. Check it's enabled: `sysctl net.ipv4.tcp_syncookies` should be 1.

Q9: You're deploying to 50 servers and SSH connections fail to some of them. How do you debug this?

```
# 1. Basic connectivity test
ping -c 1 server || echo "Host unreachable at L3"

# 2. Test SSH port specifically
nc -zv server 22 || echo "Port 22 not reachable — firewall?"

# 3. Try with verbose SSH for handshake details
ssh -vvv user@server 2>&1 | head -50
# Look for: key exchange, authentication method, refused/timeout

# 4. Check if host key changed (MITM warning)
ssh-keyscan server 2>/dev/null | ssh-keygen -lf -
# Compare fingerprint with known_hosts

# 5. Check from the server side if accessible
sudo ss -tlnp | grep :22    # Is sshd listening?
sudo journalctl -u sshd -n 50    # Any auth errors?
sudo tail -f /var/log/auth.log   # Live auth attempts

# 6. Check firewall
sudo iptables -L INPUT -n -v | grep 22
```

Q10: How would you capture and analyze a production network issue without impacting the server?

```
# Step 1: Capture to file with minimal overhead
# -nn: skip DNS resolution
# -s 96: only capture first 96 bytes (headers, not payloads)
# 'not port 22': exclude SSH noise
# -C 100: rotate files at 100MB
# -w: save to file (don't parse on the fly — less CPU)
sudo tcpdump -nn -s 96 -i eth0 -w /tmp/cap_%Y%m%d_%H%M%S.pcap \
    -C 100 'not port 22 and not port 443' &

# Step 2: Reproduce the issue
./trigger_the_problem.sh

# Step 3: Stop capture
sudo kill %1

# Step 4: Analyze offline (on your workstation, not prod)
# Copy pcap to workstation
scp server:/tmp/cap_*.pcap ./

# Analyze with tcpdump offline
tcpdump -nn -r cap_*.pcap 'tcp[tcpflags] & tcp-rst != 0'

# Or open in Wireshark for visual analysis
# Wireshark filter: tcp.flags.reset == 1

# Step 5: If issue is recurrent, use eBPF for ongoing monitoring
# tcplife shows connection duration — catch slowness without packet
capture
sudo tcplife-bpfcc | grep <app_port>
```

Document covers all 11 topics in Category 5: Networking — from basic connectivity tools through TCP internals, container networking, and eBPF observability, with real production debugging workflows, interview-ready answers, and critical gotchas throughout.